이 파트에서는 **오픈소스 생성형 AI 생태계**를 이해하고, 전체 과정을 아우르는 **개인 프로젝트**를 수행합니다.

Hugging Face 플랫폼과 Transformers 라이브러리를 활용하고, Ollama를 이용한 로컬 LLM 실행을 체험합니다. 마지막으로 지금까지 배운 모든 기술을 종합하여 개인 프로젝트를 기획, 구현, 발표합니다.

학습 목표

- Hugging Face에서 모델을 검색하고 Pipeline API로 사용할 수 있습니다.
- Ollama를 설치하고 로컬에서 LLM을 실행할 수 있습니다.
- 전체 과정에서 배운 기술을 종합하여 개인 프로젝트를 완성합니다.

이 파트의 구성

챕터	제목	핵심 내용
Chapter 24	Hugging Face와 오픈소스 모델	Transformers, Pipeline API, 모델 비교
Chapter 25	로컬 LLM 실행	Ollama, LM Studio, 로컬 vs 클라우드
Chapter 26	종합 정리와 개인 프로젝트	전체 리뷰, 트렌드, 프로젝트 발표

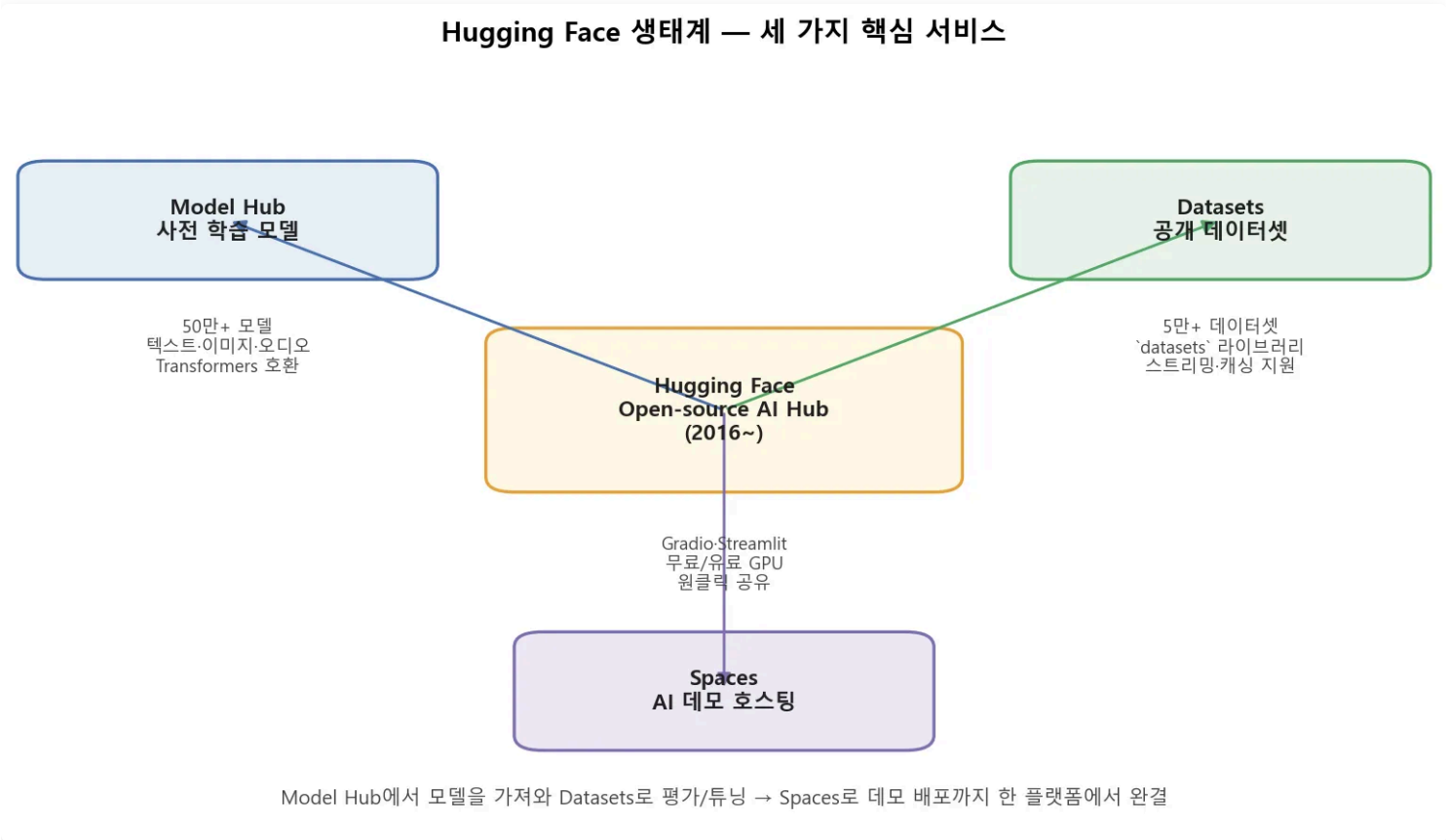
이번 챕터에서는 **Hugging Face(허깅 페이스)**와 오픈소스 LLM 생태계를 살펴봅니다. Hugging Face는 수십만 개의 사전 학습 모델과 데이터셋을 제공하는 AI 허브로, 실무에서 가장 많이 활용되는 플랫폼입니다. Pipeline API 사용법부터 주요 오픈소스 LLM 비교까지, 실전에 바로 적용할 수 있는 내용을 중심으로 다룹니다.

24.1 Hugging Face 플랫폼 소개

Hugging Face(허깅 페이스)는 NLP(자연어 처리)를 비롯한 다양한 AI 모델을 공유하고 협업하는 오픈소스 플랫폼입니다. 2016년 설립 이후 빠르게 성장하여 현재 AI 커뮤니티의 핵심 인프라로 자리잡았습니다.

주요 구성 요소

Hugging Face는 크게 세 가지 핵심 서비스로 구성됩니다.



- **Model Hub(모델 허브)**: 50만 개 이상의 사전 학습 모델이 공개되어 있으며, 텍스트·이미지·오디오 등 다양한 모달리티를 지원합니다.
- **Datasets(데이터셋)**: 수만 개의 공개 데이터셋을 제공하며, `datasets` 라이브러리로 손쉽게 로드할 수 있습니다.
- **Spaces(스페이스)**: Gradio 또는 Streamlit 기반의 AI 데모 앱을 배포하고 공유할 수 있는 호스팅 환경입니다.

왜 Hugging Face를 사용하는가

> *오픈소스 AI 모델을 처음부터 학습하는 것은 막대한 비용과 시간이 필요합니다. Hugging Face는 이미 학습된 고성능 모델을 몇 줄의 코드만으로 활용할 수 있도록 해 줍니다.*

Hugging Face를 활용하면 다음과 같은 이점이 있습니다.

- **빠른 프로토타이핑**: 검증된 모델을 즉시 사용하여 아이디어를 빠르게 검증할 수 있습니다.
- **비용 절감**: 클라우드 LLM API 비용 없이 로컬 또는 자체 서버에서 모델을 실행할 수 있습니다.
- **커스터마이징**: 공개 모델을 파인튜닝하여 도메인 특화 모델을 만들 수 있습니다.
- **투명성**: 오픈소스이므로 모델의 구조와 학습 방법을 직접 확인할 수 있습니다.

24.2 Transformers 라이브러리 사용법

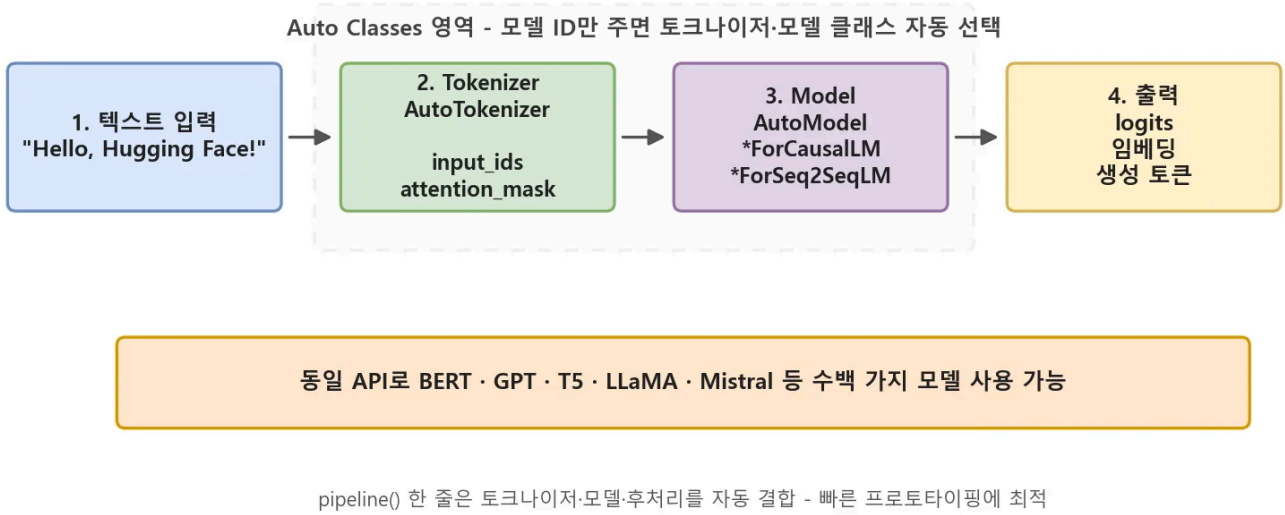
Transformers 라이브러리는 Hugging Face의 핵심 파이썬 라이브러리로, BERT, GPT, T5 등 수백 가지 트랜스포머 기반 모델을 통일된 API로 사용할 수 있게 해 줍니다.

설치

```
pip install transformers
pip install transformers[torch]    # PyTorch와 함께 설치
pip install transformers[tf]       # TensorFlow와 함께 설치
```

핵심 구성 요소

Transformers 라이브러리는 세 가지 핵심 클래스를 중심으로 동작합니다.



텍스트 입력 → `AutoTokenizer`가 토큰 ID로 변환 → `AutoModel*` 클래스가 임베딩/로짓/생성 토큰을 산출 → 후처리 결과 반환. `Auto*` 클래스는 모델 ID만으로 적합한 구현을 자동 선택하므로, 동일한 코드로 BERT·GPT·T5·LLaMA·Mistral 등 수백 가지 모델을 다룰 수 있습니다.

1. Tokenizer(토크나이저)

텍스트를 모델이 이해할 수 있는 토큰 ID 시퀀스로 변환합니다.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = "Hello, Hugging Face!"
tokens = tokenizer(text, return_tensors="pt")

print(tokens)
# {'input_ids': tensor([[101, 7592, 1010, 17662, 2227, 999, 102]]),
#  'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1]])}
```

2. Model(모델)

사전 학습된 가중치를 로드하고 추론(inference)을 수행합니다.

```
from transformers import AutoModel, AutoTokenizer
import torch

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")

inputs = tokenizer("Hello, world!", return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs)

# last_hidden_state: 각 토큰의 임베딩 벡터
print(outputs.last_hidden_state.shape)  # [1, 5, 768]
```

3. Auto Classes(오토 클래스)

`AutoTokenizer`, `AutoModel`, `AutoModelForSequenceClassification` 등의 `Auto` 클래스는 모델 이름만으로 적합한 클래스를 자동으로 선택해 줍니다.

```
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    AutoModelForSeq2SeqLM,
    AutoModelForCausalLM,
)

# 태스크별 적절한 모델 클래스 자동 선택
# 분류 모델
clf_model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased-finetuned-sst-2-english")

# 시퀀스-투-시퀀스 모델 (번역, 요약)
seq2seq_model = AutoModelForSeq2SeqLM.from_pretrained("facebook/bart-large-cnn")

# 인과적 언어 모델 (텍스트 생성)
causal_model = AutoModelForCausalLM.from_pretrained("gpt2")
```

모델 저장과 로컬 로드

네트워크 없이 사용하거나 반복 실행 시 다운로드 시간을 절약하기 위해 모델을 로컬에 저장할 수 있습니다.

```
from transformers import AutoTokenizer, AutoModel

model_name = "bert-base-uncased"
save_path = "./local_models/bert-base-uncased"

# 다운로드 및 저장
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

tokenizer.save_pretrained(save_path)
model.save_pretrained(save_path)

# 로컬에서 로드
tokenizer = AutoTokenizer.from_pretrained(save_path)
model = AutoModel.from_pretrained(save_path)
```

24.3 모델 허브에서 모델 검색 및 다운로드

Model Hub(모델 허브)에서는 웹 브라우저와 코드 두 가지 방법으로 모델을 검색하고 다운로드할 수 있습니다.

웹에서 검색하기

`https://huggingface.co/models`에 접속하면 다음 필터를 활용하여 모델을 탐색할 수 있습니다.

- **Task(태스크)**: Text Classification, Summarization, Translation, Question Answering 등
- **Library(라이브러리)**: Transformers, PyTorch, TensorFlow 등
- **Language(언어)**: 한국어(`ko`), 영어(`en`) 등 특정 언어 필터링
- **License(라이선스)**: Apache 2.0, MIT 등 상업적 이용 가능 여부 확인

`huggingface_hub` 라이브러리로 검색하기

```
from huggingface_hub import HfApi

api = HfApi()

# 한국어 텍스트 분류 모델 검색
models = api.list_models(
    filter="text-classification",
    language="ko",
    sort="downloads",
    direction=-1,   # 내림차순 (다운로드 많은 순)
    limit=10,
)

for model in models:
    print(f"{model.id} - Downloads: {model.downloads}")
```

모델 카드(Model Card) 확인

모델을 사용하기 전에 반드시 **Model Card(모델 카드)**를 확인하는 것이 좋습니다. 모델 카드에는 다음 정보가 담겨 있습니다.

- 모델의 학습 데이터와 방법론
- 성능 벤치마크 결과
- 라이선스 및 사용 제한 사항
- 편향성(bias)과 한계에 대한 설명

`snapshot_download`로 전체 모델 다운로드

```
from huggingface_hub import snapshot_download

# 모델 전체를 로컬 디렉터리에 다운로드
local_dir = snapshot_download(
    repo_id="microsoft/phi-2",
    local_dir="./models/phi-2",
    ignore_patterns=["*.msgpack", "*.h5"],  # 불필요한 파일 제외
)

print(f"모델 저장 위치: {local_dir}")
```

환경 변수로 캐시 경로 설정

기본 캐시 경로(`~/.cache/huggingface/hub`)를 변경하려면 환경 변수를 설정합니다.

```
import os

# 사용자 지정 캐시 경로 설정
os.environ["HF_HOME"] = "/data/huggingface_cache"

# 또는 .env 파일에 추가
# HF_HOME=/data/huggingface_cache
```

24.4 Pipeline API로 간편하게 모델 사용하기

Pipeline API는 복잡한 전처리·추론·후처리 과정을 한 줄의 코드로 처리할 수 있는 고수준 인터페이스입니다. 다양한 NLP 태스크를 빠르게 실험하고 프로토타이핑하는 데 최적화되어 있습니다.

Pipeline 기본 사용법

```
from transformers import pipeline

# 태스크 이름만으로 파이프라인 생성 (기본 모델 자동 선택)
classifier = pipeline("text-classification")
result = classifier("I love using Hugging Face!")
print(result)  # [{'label': 'POSITIVE', 'score': 0.9998}]
```

텍스트 분류 (Text Classification)

텍스트 분류(Text Classification)는 입력 텍스트에 카테고리 레이블을 부여하는 태스크입니다. 감성 분석이 대표적인 예입니다.

```
from transformers import pipeline

# 감성 분석
sentiment = pipeline(
    "text-classification",
    model="snunlp/KR-FinBert-SC",  # 한국어 금융 감성 분석 모델
    device=0,  # GPU 사용 (CPU는 -1)
)

texts = [
    "이 제품 정말 훌륭해요! 강력 추천합니다.",
    "배송이 너무 늦고 품질도 실망스럽네요.",
    "가격 대비 무난한 것 같습니다.",
]

results = sentiment(texts)
for text, result in zip(texts, results):
    print(f"텍스트: {text}")
    print(f"  레이블: {result['label']}, 점수: {result['score']:.4f}\n")
```

다중 레이블 분류 및 **제로샷 분류(Zero-shot Classification)**도 지원합니다.

```
# 제로샷 분류: 학습 데이터 없이 임의 레이블로 분류
zero_shot = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")

result = zero_shot(
    "오늘 코스피가 2% 상승하며 강세를 보였습니다.",
    candidate_labels=["경제", "스포츠", "정치", "문화"],
)

print(result["labels"])  # ['경제', '정치', '문화', '스포츠']
print(result["scores"])  # [0.92, 0.04, 0.03, 0.01]
```

텍스트 요약 (Summarization)

텍스트 요약(Summarization)은 긴 문서를 짧은 핵심 내용으로 압축하는 태스크입니다.

```
from transformers import pipeline

summarizer = pipeline(
    "summarization",
    model="facebook/bart-large-cnn",
    device=0,
)

article = """
The tower is 324 metres (1,063 ft) tall, about the same height as an 81-storey building,
and the tallest structure in Paris. Its base is square, measuring 125 metres (410 ft) on each
side.
During its construction, the Eiffel Tower surpassed the Washington Monument to become the tall
est
man-made structure in the world, a title it held for 41 years until the Chrysler Building in N
ew York
City was finished in 1930. It was the first structure in the world to reach a height of 300 me
tres.
Due to the addition of a broadcasting aerial at the top of the tower in 1957, it is now taller
than
the Chrysler Building by 5.2 metres (17 ft).
"""

summary = summarizer(
    article,
    max_length=130,
    min_length=30,
    do_sample=False,
)

print(summary[0]["summary_text"])
# The Eiffel Tower is 324 metres tall, about the same height as an 81-storey building.
# It is the tallest structure in Paris and was the first structure to reach 300 metres.
```

번역 (Translation)

번역(Translation)은 소스 언어 텍스트를 타겟 언어로 변환하는 태스크입니다.

```
from transformers import pipeline

# 영어 → 한국어 번역
translator = pipeline(
    "translation",
    model="Helsinki-NLP/opus-mt-en-ko",
)

texts = [
    "Artificial intelligence is transforming every industry.",
    "Machine learning models can now understand human language.",
]

translations = translator(texts)
for original, translated in zip(texts, translations):
    print(f"원문: {original}")
    print(f"번역: {translated['translation_text']}\n")
```

다국어 모델(Multilingual Model)을 사용하면 다양한 언어 쌍을 하나의 모델로 처리할 수 있습니다.

```
# Helsinki-NLP/opus-mt-tc-big-ko-en: 한국어 → 영어
ko_en_translator = pipeline(
    "translation",
    model="Helsinki-NLP/opus-mt-tc-big-ko-en",
)

result = ko_en_translator("인공지능은 미래를 바꾸고 있습니다.")
print(result[0]["translation_text"])
# Artificial intelligence is changing the future.
```

질의응답 (Question Answering)

질의응답(Question Answering, QA)은 주어진 컨텍스트(지문)에서 질문에 대한 답을 추출하는 태스크입니다.

```
from transformers import pipeline

qa = pipeline(
    "question-answering",
    model="deepset/roberta-base-squad2",
)

context = """
Hugging Face was founded in 2016 by Clément Delangue, Julien Chaumond, and Thomas Wolf.
The company is headquartered in New York City and Paris. Hugging Face is known for its
Transformers library, which provides thousands of pretrained models for natural language
processing, computer vision, and audio tasks.
"""

questions = [
    "When was Hugging Face founded?",
    "Where is Hugging Face headquartered?",
    "What is Hugging Face known for?",
]

for question in questions:
    result = qa(question=question, context=context)
    print(f"질문: {question}")
    print(f"답변: {result['answer']} (신뢰도: {result['score']:.4f})\n")
```

텍스트 생성 (Text Generation)

텍스트 생성(Text Generation)은 입력 프롬프트를 이어받아 자연스러운 텍스트를 생성하는 태스크입니다.

```
from transformers import pipeline

generator = pipeline(
    "text-generation",
    model="gpt2",
    device=0,
)

prompt = "Once upon a time in a land far away,"

outputs = generator(
    prompt,
    max_new_tokens=100,
    num_return_sequences=2,    # 2개의 다양한 결과 생성
    temperature=0.8,           # 창의성 조절 (높을수록 다양)
    do_sample=True,
    pad_token_id=50256,
)

for i, output in enumerate(outputs, 1):
    print(f"[결과 {i}]")
    print(output["generated_text"])
    print()
```

Pipeline 공통 매개변수

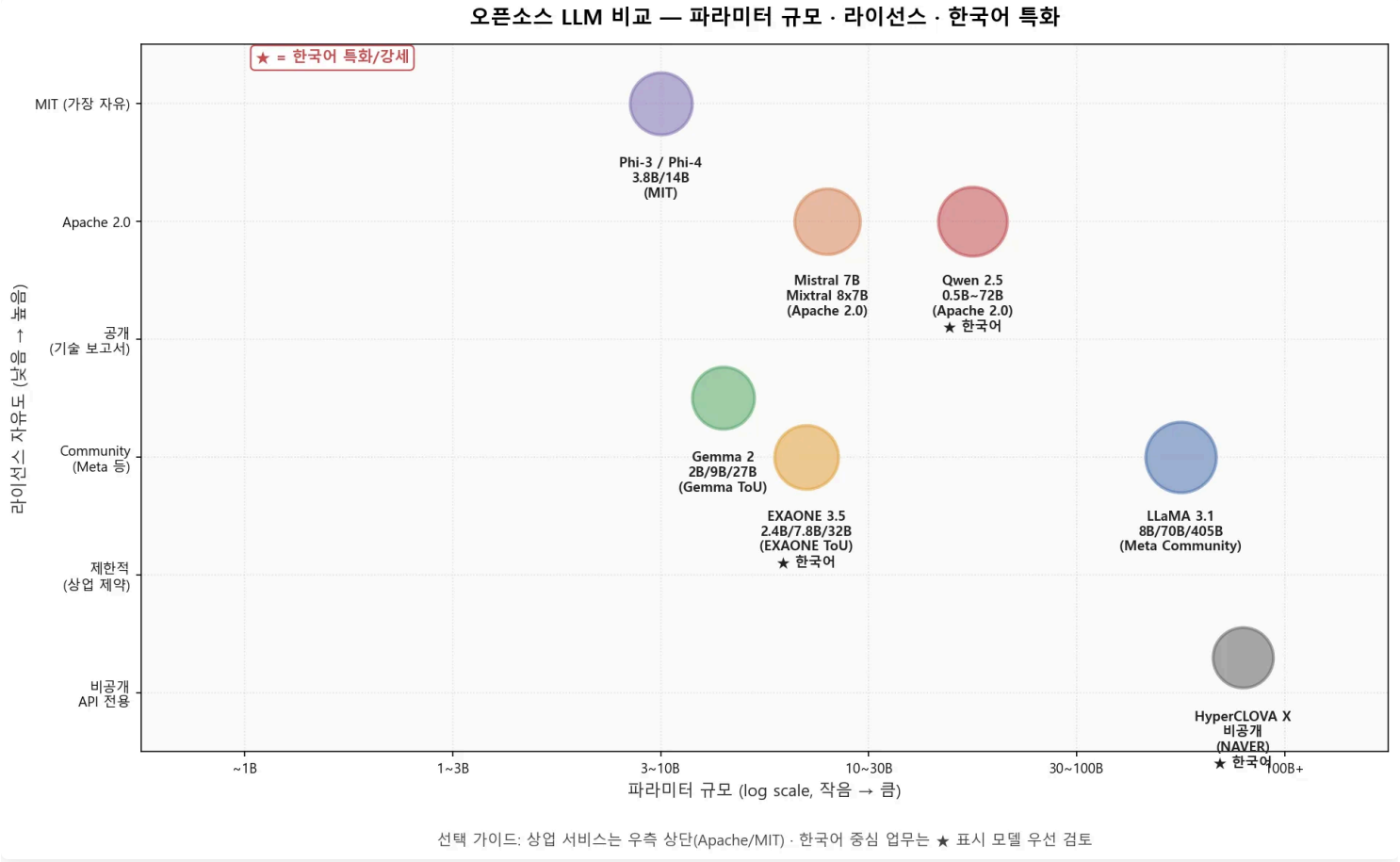
매개변수	설명	기본값
`model`	사용할 모델 이름 또는 경로	태스크별 기본 모델
`device`	실행 디바이스 (0=GPU, -1=CPU)	-1
`batch_size`	배치 크기 (처리 속도 향상)	1
`max_length`	출력 최대 토큰 수	모델별 상이
`truncation`	입력이 최대 길이 초과 시 자르기	False

24.5 오픈소스 LLM 비교

오픈소스 **LLM(Large Language Model, 대규모 언어 모델)** 생태계는 빠르게 발전하고 있습니다. 각 모델의 특성을 이해하면 사용 목적에 맞는 모델을 효과적으로 선택할 수 있습니다.

주요 오픈소스 LLM 비교

모델	개발사	파라미터	라이선스	특징
LLaMA 3.1	Meta	8B / 70B / 405B	Llama 3.1 Community	강력한 범용 성능, 다국어 지원
Mistral / Mixtral	Mistral AI	7B / 8x7B (MoE)	Apache 2.0	효율적인 아키텍처, 상업적 이용 가능
Gemma 2	Google DeepMind	2B / 9B / 27B	Gemma ToU	경량·고성능, 모바일 친화적
Phi-3 / Phi-4	Microsoft	3.8B / 14B	MIT	소형 모델 대비 높은 추론 능력
Qwen 2.5	Alibaba Cloud	0.5B ~ 72B	Apache 2.0	한국어·중국어 포함 강력한 다국어
EXAONE 3.5	LG AI Research	2.4B / 7.8B / 32B	EXAONE AI Model License	한국어 특화, 국내 기업 개발
HyperCLOVA X	NAVER	비공개	비공개	한국어 최고 수준, API 제공
Llama 4 (Scout/Maverick/Behemoth)	Meta, 2025.4	17B~2T MoE	Llama 4 Community	멀티모달 네이티브, 10M+ 컨텍스트, 오픈 가중치
Qwen 3	Alibaba, 2025	다양 (소형~대형)	Apache 2.0	Thinking 모드, 다국어·한국어 강점
DeepSeek V3 / R1	DeepSeek, 2024.12 / 2025.1	671B MoE (V3)	DeepSeek License	V3는 범용·코딩, R1은 추론 특화 (o1급)
Phi-4 / Phi-4-multimodal	Microsoft, 2024.12~2025	14B / 5B	MIT	소형 추론 SOTA, 멀티모달 변형



> 💡 ***2026년 시점 추가 동향**: DeepSeek-R1·QwQ 같은 **추론 특화 오픈모델**, Llama 4·Qwen 3 같은 **네이티브 멀티모달 오픈모델**이 빠르게 자리 잡았습니다. 한국어 모델은 EXAONE 4·HyperCLOVA X v2 등 후속 버전이 등장 가능하니 각 사 공식 페이지 확인을 권장합니다.*

가로축은 파라미터 규모, 세로축은 라이선스 자유도입니다. 상업 서비스라면 우측 상단의 **Apache 2.0/MIT** 라이선스 모델(Mistral·Qwen·Phi)을, 한국어가 핵심이라면 ★ 표시된 **Qwen 2.5 · EXAONE · HyperCLOVA X**를 우선 검토하세요.

LLaMA (라마)

LLaMA(Large Language Model Meta AI)는 Meta에서 개발한 오픈소스 LLM 시리즈입니다.

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

model_id = "meta-llama/Meta-Llama-3.1-8B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.bfloat16,   # 메모리 절약
    device_map="auto",             # GPU/CPU 자동 배분
)

messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "파이썬에서 리스트 컴프리헨션이란 무엇인가요?"},
]

input_ids = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    return_tensors="pt",
).to(model.device)

outputs = model.generate(
    input_ids,
    max_new_tokens=512,
    temperature=0.7,
    do_sample=True,
)

response = tokenizer.decode(outputs[0][input_ids.shape[-1]:], skip_special_tokens=True)
print(response)
```

> *LLaMA 3.1 모델은 Hugging Face에서 다운로드하려면 Meta의 라이선스 동의가 필요합니다. `huggingface-cli login` 명령으로 토큰을 설정한 후 사용하세요.*

Mistral / Mixtral (미스트랄 / 믹스트랄)

Mistral 7B은 7B 파라미터임에도 13B 모델에 필적하는 성능을 보이며 Apache 2.0 라이선스로 상업적 이용이 가능합니다. **Mixtral 8x7B**은 **MoE(Mixture of Experts)** 아키텍처를 사용하여 8개의 전문가 네트워크 중 2개를 선택적으로 활성화함으로써 효율성을 극대화합니다.

```
from transformers import pipeline
import torch

# Mistral Instruct 모델로 대화
pipe = pipeline(
    "text-generation",
    model="mistralai/Mistral-7B-Instruct-v0.3",
    torch_dtype=torch.bfloat16,
    device_map="auto",
)

messages = [{"role": "user", "content": "머신러닝과 딥러닝의 차이점을 설명해 주세요."}]

output = pipe(messages, max_new_tokens=256)
print(output[0]["generated_text"][-1]["content"])
```

Gemma (젬마)

Gemma는 Google DeepMind가 Gemini 기술을 기반으로 개발한 경량 오픈소스 모델입니다. 2B, 9B, 27B 크기로 제공되며 특히 소형 모델에서 뛰어난 성능을 발휘합니다.

```
from transformers import pipeline
import torch

pipe = pipeline(
    "text-generation",
    model="google/gemma-2-9b-it",
    torch_dtype=torch.bfloat16,
    device_map="auto",
)

output = pipe(
    "파이썬으로 피보나치 수열을 구하는 함수를 작성해 주세요.",
    max_new_tokens=512,
)
print(output[0]["generated_text"])
```

Phi (파이)

Phi는 Microsoft Research에서 개발한 소형 언어 모델 시리즈입니다. **Phi-4**는 14B 파라미터로 GPT-3.5 수준의 추론 능력을 달성했으며, MIT 라이선스로 완전한 상업적 활용이 가능합니다.

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

model_id = "microsoft/phi-4"

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)

inputs = tokenizer(
    "◁user▷\n수학 문제를 풀어 주세요: 2x + 5 = 15, x는?\n◁assistant▷\n",
    return_tensors="pt",
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=256)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

Qwen (치엔)

Qwen(Qianwen, 千問)은 Alibaba Cloud에서 개발한 다국어 LLM 시리즈입니다. **Qwen 2.5**는 0.5B부터 72B까지 다양한 크기로 제공되며, 한국어를 포함한 29개 언어에서 뛰어난 성능을 보입니다.

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "Qwen/Qwen2.5-7B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype="auto",
    device_map="auto",
)

messages = [
    {"role": "system", "content": "당신은 유용한 AI 어시스턴트입니다."},
    {"role": "user", "content": "한국의 전통 음식 5가지를 소개해 주세요."},
]

text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
)

inputs = tokenizer([text], return_tensors="pt").to(model.device)
outputs = model.generate(**inputs, max_new_tokens=512)

response = tokenizer.decode(outputs[0][inputs.input_ids.shape[-1]:], skip_special_tokens=True)
print(response)
```

모델 선택 기준

모델을 선택할 때는 다음 기준을 고려하는 것이 좋습니다.

- **하드웨어 제약**: GPU VRAM이 8GB라면 7B 모델(4-bit 양자화 시), 24GB라면 13~34B 모델을 사용할 수 있습니다.
- **언어 요구사항**: 한국어가 중요하다면 Qwen 2.5, EXAONE, HyperCLOVA X를 우선 검토하세요.
- **라이선스**: 상업적 서비스에는 Apache 2.0 또는 MIT 라이선스 모델(Mistral, Qwen, Phi)을 권장합니다.
- **태스크 특성**: 코딩은 DeepSeek-Coder나 CodeLlama, 수학은 Qwen2.5-Math, 범용은 LLaMA 3.1이 적합합니다.

[실습] Hugging Face Pipeline으로 텍스트 분류/요약 실행

이번 실습에서는 Hugging Face Pipeline API를 활용하여 한국어 텍스트의 분류와 요약을 직접 실행해 봅니다.

실습 환경 준비

```
# 필요 라이브러리 설치
# pip install transformers torch sentencepiece

from transformers import pipeline
import torch

# GPU 사용 가능 여부 확인
device = 0 if torch.cuda.is_available() else -1
print(f"사용 디바이스: {'GPU' if device == 0 else 'CPU'}")
```

실습 1: 텍스트 감성 분류

```
from transformers import pipeline

# 다국어 감성 분석 파이프라인 생성
sentiment_pipeline = pipeline(
    "text-classification",
    model="snunlp/KR-FinBert-SC",
    device=device,
)

# 분류할 한국어 텍스트 목록
sample_texts = [
    "오늘 회의에서 새로운 프로젝트가 승인되어 기쁩니다.",
    "예상보다 실적이 저조하여 주가가 하락했습니다.",
    "내일 날씨는 맑고 기온은 평년과 비슷할 전망입니다.",
    "고객 서비스 응대가 너무 불친절하고 실망스러웠습니다.",
    "신제품 출시 이후 매출이 꾸준히 증가하고 있습니다.",
]

print("=" * 60)
print("텍스트 감성 분류 결과")
print("=" * 60)

results = sentiment_pipeline(sample_texts)
for text, result in zip(sample_texts, results):
    label = result["label"]
    score = result["score"]
    print(f"\n텍스트: {text[:30]}...")
    print(f"  레이블: {label}")
    print(f"  신뢰도: {score:.4f} ({score*100:.1f}%)")
```

실습 2: 영어 텍스트 요약

```
from transformers import pipeline

# 텍스트 요약 파이프라인 생성
summarizer = pipeline(
    "summarization",
    model="facebook/bart-large-cnn",
    device=device,
)

# 요약할 뉴스 기사 (영어)
news_articles = [
    """
    Scientists have discovered a new species of deep-sea fish in the Pacific Ocean
    at a depth of over 8,000 meters. The fish, named Pseudoliparis swirei, was found
    in the Mariana Trench and is notable for its translucent body and unique adaptations
    to extreme pressure. The discovery was made during an international research expedition
    that used advanced deep-sea submersibles equipped with high-resolution cameras.
    Researchers believe this discovery could provide valuable insights into how life
    adapts to extreme environments and might help in developing new pressure-resistant
    materials for industrial applications.
    """,
    """
    A breakthrough in renewable energy storage has been announced by researchers at
    MIT. The team developed a new type of battery using molten salt that can store
    electricity for weeks at a fraction of the cost of lithium-ion batteries.
    This technology could revolutionize the way we store energy from solar and wind
    sources, making renewable energy more viable for large-scale grid applications.
    The battery operates at high temperatures, which actually improves its efficiency
    and longevity compared to conventional batteries.
    """,
]

print("=" * 60)
print("텍스트 요약 결과")
print("=" * 60)

for i, article in enumerate(news_articles, 1):
    summary = summarizer(
        article.strip(),
        max_length=80,
```

```
        min_length=20,
        do_sample=False,
    )
    print(f"\n[기사 {i}]")
    print(f"원문 길이: {len(article.split())} 단어")
    print(f"요약: {summary[0]['summary_text']}")
    print(f"요약 길이: {len(summary[0]['summary_text'].split())} 단어")
```

실습 3: 질의응답 파이프라인

```
from transformers import pipeline

# 질의응답 파이프라인
qa_pipeline = pipeline(
    "question-answering",
    model="deepset/roberta-base-squad2",
    device=device,
)

# 컨텍스트와 질문 설정
context = """
Hugging Face is an AI company founded in 2016 by Clément Delangue, Julien Chaumond,
and Thomas Wolf. The company is headquartered in New York City with offices in Paris.
Hugging Face is best known for developing the Transformers library, which has become
the standard toolkit for working with large language models. The platform hosts over
500,000 models, 50,000 datasets, and 100,000 demo apps (Spaces). In 2023, the company
raised $235 million in Series D funding, valuing it at $4.5 billion.
"""

questions = [
    "When was Hugging Face founded?",
    "Who are the founders of Hugging Face?",
    "How many models are hosted on Hugging Face?",
    "How much did Hugging Face raise in Series D?",
]

print("=" * 60)
print("질의응답 결과")
print("=" * 60)

for question in questions:
    result = qa_pipeline(question=question, context=context)
    print(f"\n질문: {question}")
    print(f"답변: {result['answer']}")
    print(f"신뢰도: {result['score']:.4f}")
```

실습 4: 파이프라인 통합 활용

```
from transformers import pipeline

def analyze_text(text: str, lang: str = "ko") → dict:
    """텍스트를 분석하여 감성, 키워드 등을 반환하는 함수."""

    results = {}

    # 1단계: 제로샷 분류로 주제 파악
    topic_classifier = pipeline(
        "zero-shot-classification",
        model="facebook/bart-large-mnli",
        device=-1,
    )

    topic_result = topic_classifier(
        text,
        candidate_labels=["경제/비즈니스", "기술/IT", "사회/문화", "스포츠", "정치"],
    )
    results["topic"] = topic_result["labels"][0]
    results["topic_score"] = round(topic_result["scores"][0], 4)

    # 2단계: 영문 번역 후 요약 (한국어 모델 제한 극복)
    if lang == "ko":
        translator = pipeline(
            "translation",
            model="Helsinki-NLP/opus-mt-ko-en",
            device=-1,
        )
        translated = translator(text)[0]["translation_text"]
    else:
        translated = text

    summarizer = pipeline("summarization", model="facebook/bart-large-cnn", device=-1)
    if len(translated.split()) > 30:
        summary = summarizer(translated, max_length=50, min_length=10, do_sample=False)
        results["summary"] = summary[0]["summary_text"]
    else:
        results["summary"] = translated

    return results
```

```
# 테스트 실행
test_text = """
삼성전자가 3나노 공정 기반의 차세대 AI 반도체 개발에 성공했다고 발표했습니다.
이 칩은 기존 대비 30% 향상된 성능과 25% 낮은 전력 소비를 실현하며,
2025년 하반기부터 스마트폰과 데이터센터용 AI 가속기에 탑재될 예정입니다.
"""

print("텍스트 통합 분석 시작...")
analysis = analyze_text(test_text, lang="ko")
print(f"\n주제: {analysis['topic']} (신뢰도: {analysis['topic_score']})")
print(f"요약: {analysis['summary']}")
```

실습 결과 해석 가이드

실습을 통해 다음 사항을 확인합니다.

- **신뢰도 점수**: 0.9 이상이면 높은 신뢰도, 0.7 미만이면 결과를 재검토하는 것이 좋습니다.
- **모델 선택**: 한국어 성능은 모델마다 크게 다릅니다. 실제 서비스 전에 반드시 한국어 데이터로 평가하세요.
- **배치 처리**: 여러 텍스트를 한 번에 처리할 때는 리스트를 전달하여 처리 속도를 향상시킵니다.
- **GPU 활용**: `device=0`으로 GPU를 사용하면 CPU 대비 10~50배 빠른 처리가 가능합니다.

정리

이번 챕터에서는 Hugging Face 플랫폼과 오픈소스 LLM 생태계의 핵심을 살펴보았습니다.

- **Hugging Face**는 Model Hub, Datasets, Spaces로 구성된 AI 허브로, 50만 개 이상의 모델을 제공합니다.
- **Transformers 라이브러리**의 `AutoTokenizer`, `AutoModel`, Auto 클래스를 활용하면 다양한 모델을 통일된 방식으로 사용할 수 있습니다.
- **Pipeline API**는 텍스트 분류, 요약, 번역, 질의응답 등 복잡한 태스크를 몇 줄의 코드로 처리합니다.
- **LLaMA, Mistral, Gemma, Phi, Qwen** 등 각 오픈소스 LLM은 크기, 라이선스, 언어 지원, 특화 분야에서 차이가 있으므로 목적에 맞게 선택해야 합니다.

다음 챕터에서는 이러한 오픈소스 모델을 로컬 환경에서 직접 실행하는 방법을 더 깊이 알아보겠습니다.

이전 챕터	다음 챕터
Ch.23 배포와 운영	Ch.25 로컬 LLM 실행

이 챕터에서는 클라우드 API에 의존하지 않고 자신의 컴퓨터에서 직접 **LLM(Large Language Model)**을 실행하는 방법을 배웁니다. **Ollama**와 **LM Studio** 같은 도구를 활용해 로컬 환경에서 강력한 AI 모델을 구동하고, 기존 애플리케이션과 연동하는 실전 기술을 익힙니다.

> ***이전 챕터**: Ch.24 Hugging Face 모델 활용 **다음 챕터**: Ch.26 종합 정리와 프로젝트*

25.1 로컬 LLM 실행의 의의 — 비용, 프라이버시, 오프라인

로컬 LLM(Local LLM)이란 OpenAI, Anthropic 등 외부 서버를 거치지 않고 자신의 하드웨어에서 직접 실행하는 언어 모델을 말합니다. 클라우드 API가 편리하지만, 로컬 실행이 더 나은 선택인 경우가 분명히 존재합니다.

비용 절감

클라우드 API는 **토큰(token)** 단위로 과금됩니다. 프로토타이핑 단계나 대용량 텍스트를 반복 처리할 때는 비용이 빠르게 누적됩니다. 로컬 LLM은 초기 하드웨어 비용만 지불하면 이후 사용량에 따른 추가 비용이 없습니다.

프라이버시와 보안

금융 데이터, 의료 기록, 내부 문서 등 민감한 정보를 외부 서버로 전송할 수 없는 경우가 많습니다. 로컬 LLM은 데이터가 자신의 장비를 벗어나지 않으므로 **데이터 컴플라이언스(data compliance)** 요구사항을 충족하기 훨씬 유리합니다.

오프라인 환경

인터넷이 불안정하거나 차단된 환경(보안 시설, 원격지, 항공기 내 등)에서도 AI 기능을 제공할 수 있습니다. 한 번 모델을 내려받으면 네트워크 없이도 완전하게 동작합니다.

모델 커스터마이징

클라우드 API는 제공자가 정해놓은 모델만 사용할 수 있지만, 로컬 환경에서는 **파인튜닝(fine-tuning)**된 전용 모델이나 특수 목적 모델을 자유롭게 교체할 수 있습니다.

> ***정리**: 로컬 LLM은 비용, 보안, 가용성, 유연성 네 가지 측면에서 클라우드 API와 상호 보완적인 관계입니다. 두 방식을 상황에 맞게 선택하는 것이 핵심입니다.*

25.2 하드웨어 요구사항 (GPU, RAM, VRAM)

로컬 LLM의 성능은 하드웨어에 직결됩니다. 모델 크기와 실행 방식에 따라 요구사항이 크게 달라집니다.

모델 크기와 파라미터

파라미터(parameter) 수는 모델 크기를 나타내는 주요 지표입니다. 일반적으로 다음과 같이 분류합니다.

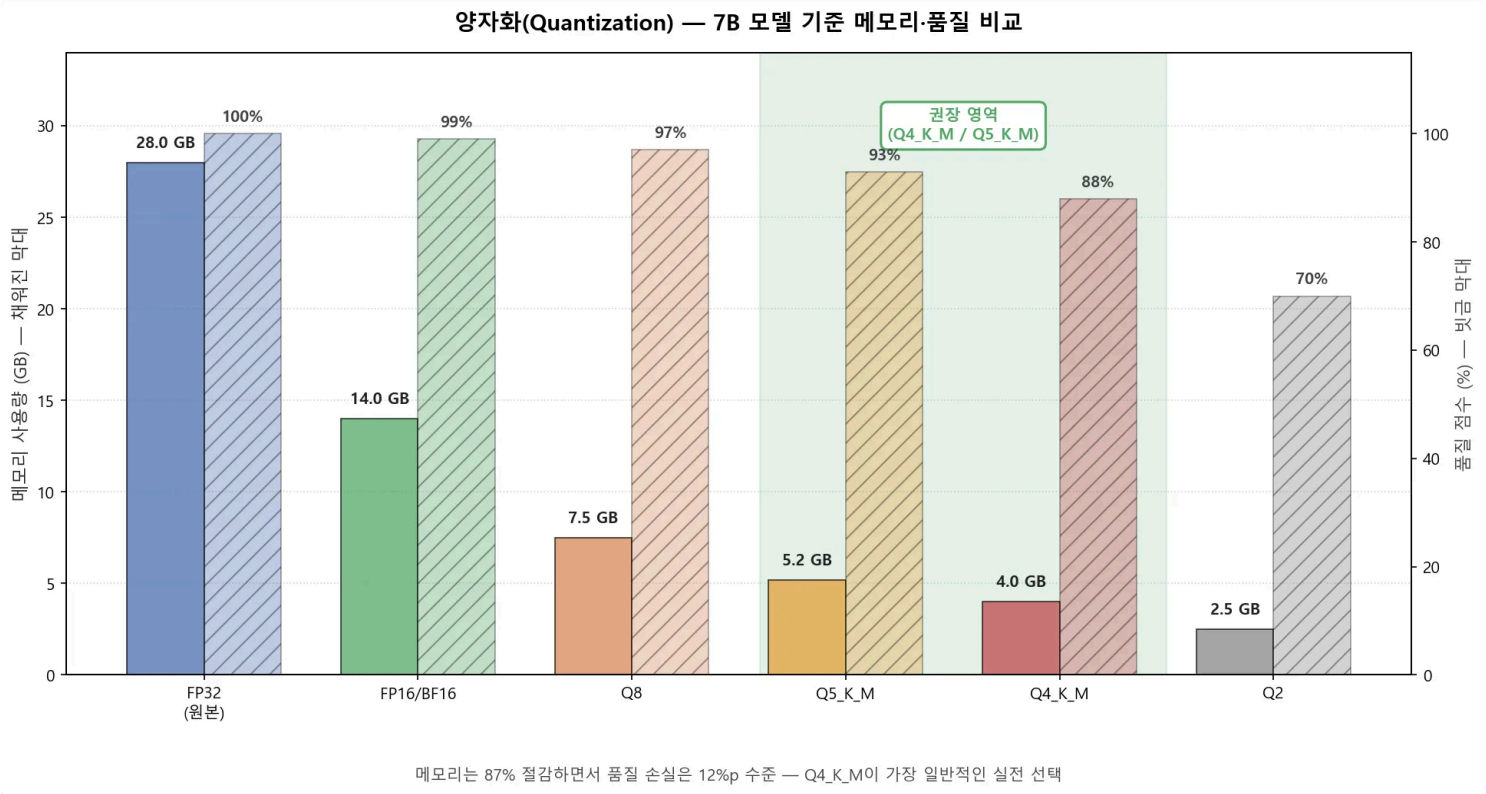
모델 규모	파라미터 수	대표 모델	최소 VRAM
소형	1B ~ 3B	Phi-3 Mini, Gemma 2B	4 GB
중형	7B ~ 13B	Llama 3.1 8B, Mistral 7B	8 GB
대형	30B ~ 70B	Llama 3.1 70B, Qwen 72B	24 GB 이상
초대형	100B 이상	Llama 3.1 405B	80 GB 이상

양자화(Quantization)

양자화(quantization)는 모델의 가중치를 더 낮은 정밀도로 압축해 메모리 사용량을 줄이는 기법입니다. 품질을 약간 희생하는 대신 훨씬 적은 VRAM으로 큰 모델을 실행할 수 있습니다.

양자화 형식	설명	메모리 절감	품질 손실
FP32	32비트 부동소수점 (원본)	기준	없음
FP16 / BF16	16비트 부동소수점	~50%	거의 없음
GGUF Q8	8비트 양자화	~75%	매우 낮음
GGUF Q4	4비트 양자화	~87%	낮음
GGUF Q2	2비트 양자화	~93%	중간

> ***실용 팁**: 일반적으로 **Q4_K_M** 또는 **Q5_K_M** 형식이 품질과 메모리 효율의 균형이 가장 좋습니다.*



7B 모델을 기준으로 양자화를 적용하면 FP32 대비 메모리를 최대 87% 절감(28GB → 4GB)할 수 있으며, 품질 손실은 약 12%p 수준에 그칩니다. 8GB VRAM에서 7B 모델을 실행하려면 **Q4_K_M**, 12GB 이상이면 **Q5_K_M**을 권장합니다.

CPU vs GPU 실행

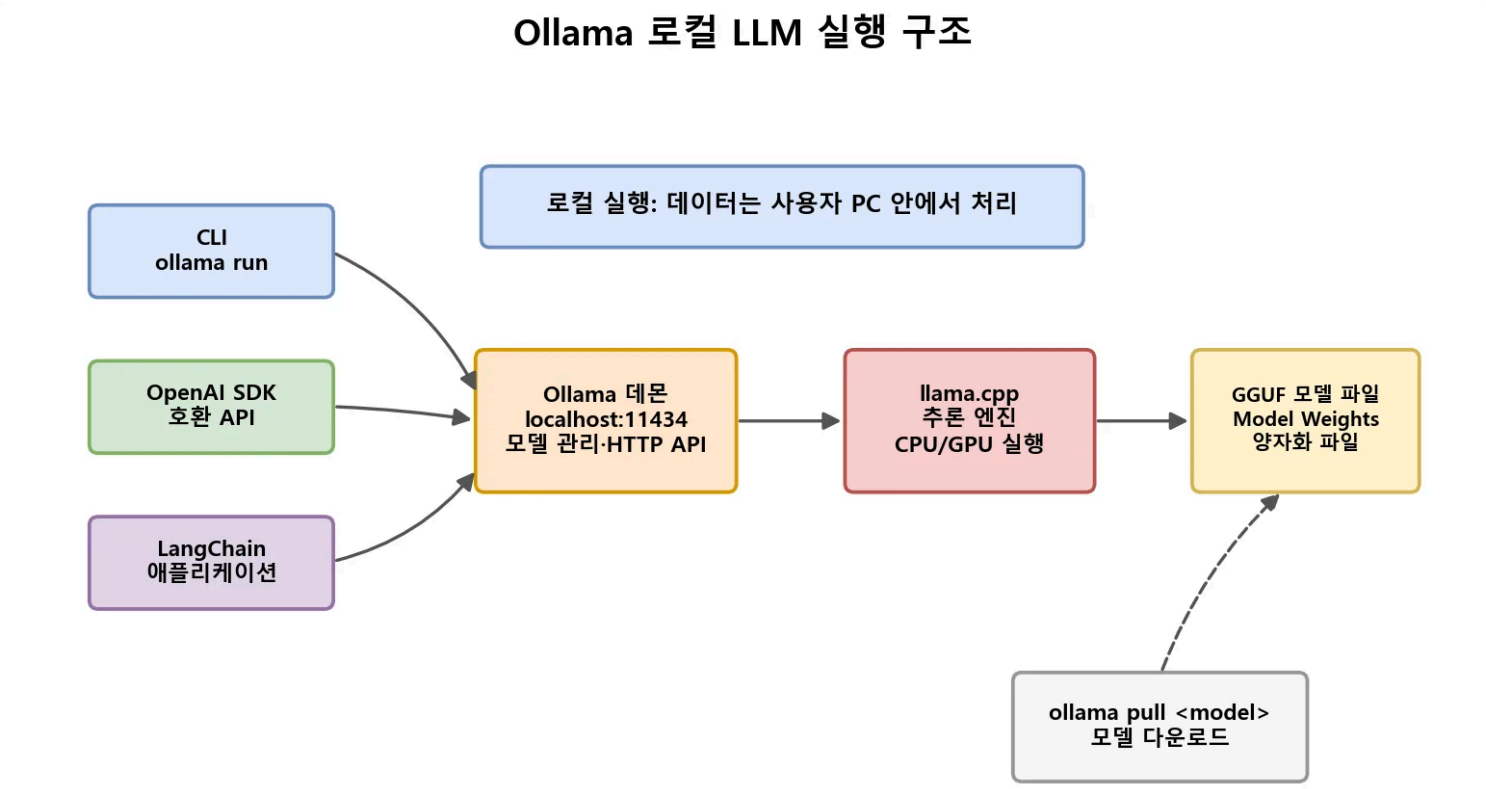
- **GPU 실행**: **VRAM(Video RAM)**에 모델을 올려 병렬 처리하므로 속도가 빠릅니다. NVIDIA GPU는 **CUDA**, AMD GPU는 **ROCm**을 활용합니다.
- **CPU 실행**: GPU가 없어도 시스템 RAM을 이용해 실행 가능합니다. 속도는 느리지만 접근성이 높습니다.
- **하이브리드**: 일부 레이어는 GPU VRAM에, 나머지는 RAM에 올리는 혼합 방식도 지원합니다.

권장 최소 사양

목적	RAM	GPU VRAM	권장 모델
입문/학습	16 GB	불필요 (CPU)	Phi-3 Mini 3.8B Q4
실용적 사용	32 GB	8 GB	Llama 3.1 8B Q4
업무급	64 GB	24 GB	Mistral 22B Q4

25.3 Ollama 설치 및 모델 다운로드

Ollama는 로컬 LLM을 가장 쉽게 실행할 수 있게 해주는 오픈소스 도구입니다. 복잡한 환경 설정 없이 단 몇 개의 명령어로 다양한 모델을 내려받고 실행할 수 있습니다.



`ollama run` CLI, OpenAI SDK, LangChain 등 어떤 클라이언트가 호출하든 모두 **Ollama 데몬(** `ollama serve` **,** `localhost:11434` **)**으로 들어가고, 데몬은 내부적으로 **llama.cpp** 추론 엔진과 **GGUF 모델 파일**을 결합해 응답을 만들어냅니다. `/v1` 엔드포인트가 OpenAI 호환이라 기존 SDK 코드는 `base_url` 만 바꾸면 그대로 작동합니다.

설치

운영체제별 설치 방법은 다음과 같습니다.

macOS / Linux

```
curl -fsSL https://ollama.com/install.sh | sh
```

Windows

Ollama 공식 사이트(https://ollama.com)에서 Windows 설치 파일을 내려받아 실행합니다.

설치 후 버전을 확인합니다.

```
ollama --version
```

모델 다운로드 및 실행

`ollama run` 명령 하나로 모델 다운로드와 실행을 동시에 처리합니다.

```
# Llama 3.2 3B 모델 실행 (최초 실행 시 자동 다운로드)
ollama run llama3.2

# 더 작은 모델로 시작하려면
ollama run phi3:mini

# 특정 양자화 버전 지정
ollama run llama3.1:8b-instruct-q4_K_M
```

모델 관리 명령어

```
# 설치된 모델 목록 확인
ollama list

# 모델 미리 다운로드 (실행하지 않고 저장만)
ollama pull mistral

# 모델 삭제
ollama rm llama3.2

# 실행 중인 모델 확인
ollama ps
```

주요 모델 목록

모델명	특징	크기	명령어
llama3.2	Meta의 최신 범용 모델	2GB	`ollama run llama3.2`
mistral	빠르고 효율적인 7B 모델	4.1GB	`ollama run mistral`
phi3:mini	Microsoft의 소형 고성능 모델	2.3GB	`ollama run phi3:mini`
gemma2	Google의 경량 모델	5.4GB	`ollama run gemma2`
qwen2.5	Alibaba의 다국어 모델	4.7GB	`ollama run qwen2.5`
codellama	코드 특화 모델	3.8GB	`ollama run codellama`
llama3.3	Meta 최신 70B 추론 강화	40GB	`ollama run llama3.3`
qwen3	Alibaba 최신, Thinking 모드	4~30GB	`ollama run qwen3`
phi4	Microsoft 14B 추론 SOTA	9GB	`ollama run phi4`
deepseek-r1	추론 특화 오픈모델 (o1급)	4~40GB	`ollama run deepseek-r1`

> ***한국어 성능 팁**: 한국어 처리가 중요하다면 **qwen2.5** 또는 **EEVE-Korean** 계열 모델을 추천합니다.*

25.4 Ollama로 대화하기 및 API 서버 활용

터미널에서 직접 대화

`ollama run` 명령 실행 후 프롬프트가 나타나면 바로 대화할 수 있습니다.

```
ollama run llama3.2
>>> 안녕하세요! 파이썬에 대해 설명해주세요.

파이썬(Python)은 1991년 귀도 반 로섬이 개발한 고수준 프로그래밍 언어입니다...

>>> /bye   # 종료
```

유용한 터미널 명령어

```
# 멀티라인 입력 모드
>>> """
... 여러 줄에 걸친
... 긴 프롬프트를 입력할 때
... """

# 시스템 프롬프트 설정
>>> /system 당신은 파이썬 전문가입니다. 항상 코드 예시를 포함해 답변하세요.

# 현재 모델 정보 확인
>>> /show info
```

REST API 서버

Ollama는 백그라운드에서 **REST API** 서버를 자동으로 실행합니다. 기본 주소는 `http://localhost:11434` 입니다.

```
# 서버 상태 확인
curl http://localhost:11434

# 설치된 모델 목록 (API)
curl http://localhost:11434/api/tags
```

Python으로 API 호출

```
import requests
import json

def chat_with_ollama(prompt: str, model: str = "llama3.2") → str:
    """Ollama REST API를 통해 모델과 대화합니다."""
    url = "http://localhost:11434/api/generate"

    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False  # 스트리밍 비활성화로 전체 응답 한 번에 수신
    }

    response = requests.post(url, json=payload)
    response.raise_for_status()

    return response.json()["response"]

# 사용 예시
answer = chat_with_ollama("파이썬 리스트와 튜플의 차이점은?")
print(answer)
```

스트리밍 응답 처리

```
import requests
import json

def stream_chat(prompt: str, model: str = "llama3.2"):
    """응답을 실시간으로 스트리밍합니다."""
    url = "http://localhost:11434/api/generate"

    payload = {
        "model": model,
        "prompt": prompt,
        "stream": True  # 스트리밍 활성화
    }

    with requests.post(url, json=payload, stream=True) as response:
        for line in response.iter_lines():
            if line:
                chunk = json.loads(line)
                print(chunk["response"], end="", flush=True)
                if chunk.get("done"):
                    break
    print()  # 줄바꿈

# 실행
stream_chat("머신러닝과 딥러닝의 차이를 설명해주세요.")
```

OpenAI 호환 API

Ollama는 **OpenAI API** 형식도 지원합니다. 기존 OpenAI SDK 코드를 거의 수정 없이 재사용할 수 있습니다.

```
from openai import OpenAI

# OpenAI 클라이언트를 Ollama 서버로 연결
client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama"  # 로컬이므로 임의 값 사용
)

response = client.chat.completions.create(
    model="llama3.2",
    messages=[
        {"role": "system", "content": "당신은 친절한 AI 어시스턴트입니다."},
        {"role": "user", "content": "파이썬으로 피보나치 수열을 구현해주세요."}
    ]
)

print(response.choices[0].message.content)
```

25.5 LM Studio 소개와 활용

LM Studio는 그래픽 사용자 인터페이스(**GUI**)를 제공하는 로컬 LLM 실행 도구입니다. 터미널이 익숙하지 않은 사용자나 다양한 모델을 시각적으로 탐색하고 싶을 때 유용합니다.

주요 특징

- **허깅페이스(Hugging Face)** 모델을 GUI 내에서 직접 검색하고 다운로드
- **GGUF** 포맷 모델 기본 지원
- 채팅 인터페이스와 프롬프트 편집기 내장
- **OpenAI 호환 로컬 서버** 기능 제공
- Windows, macOS, Linux 모두 지원

설치 및 시작

1. LM Studio 공식 사이트(https://lmstudio.ai)에서 설치 파일을 내려받습니다.
2. 설치 후 실행하면 **Discover** 탭에서 모델을 검색할 수 있습니다.
3. 원하는 모델의 양자화 버전을 선택해 다운로드합니다.
4. **Chat** 탭에서 다운로드된 모델을 불러와 대화를 시작합니다.

로컬 서버 활성화

LM Studio의 **Local Server** 탭에서 서버를 시작하면 `http://localhost:1234`로 API 요청을 받을 수 있습니다.

```
from openai import OpenAI

# LM Studio 로컬 서버 연결
client = OpenAI(
    base_url="http://localhost:1234/v1",
    api_key="lm-studio"  # 임의 값
)

response = client.chat.completions.create(
    model="lmstudio-community/Meta-Llama-3.1-8B-Instruct-GGUF",
    messages=[
        {"role": "user", "content": "안녕하세요!"}
    ],
    temperature=0.7
)

print(response.choices[0].message.content)
```

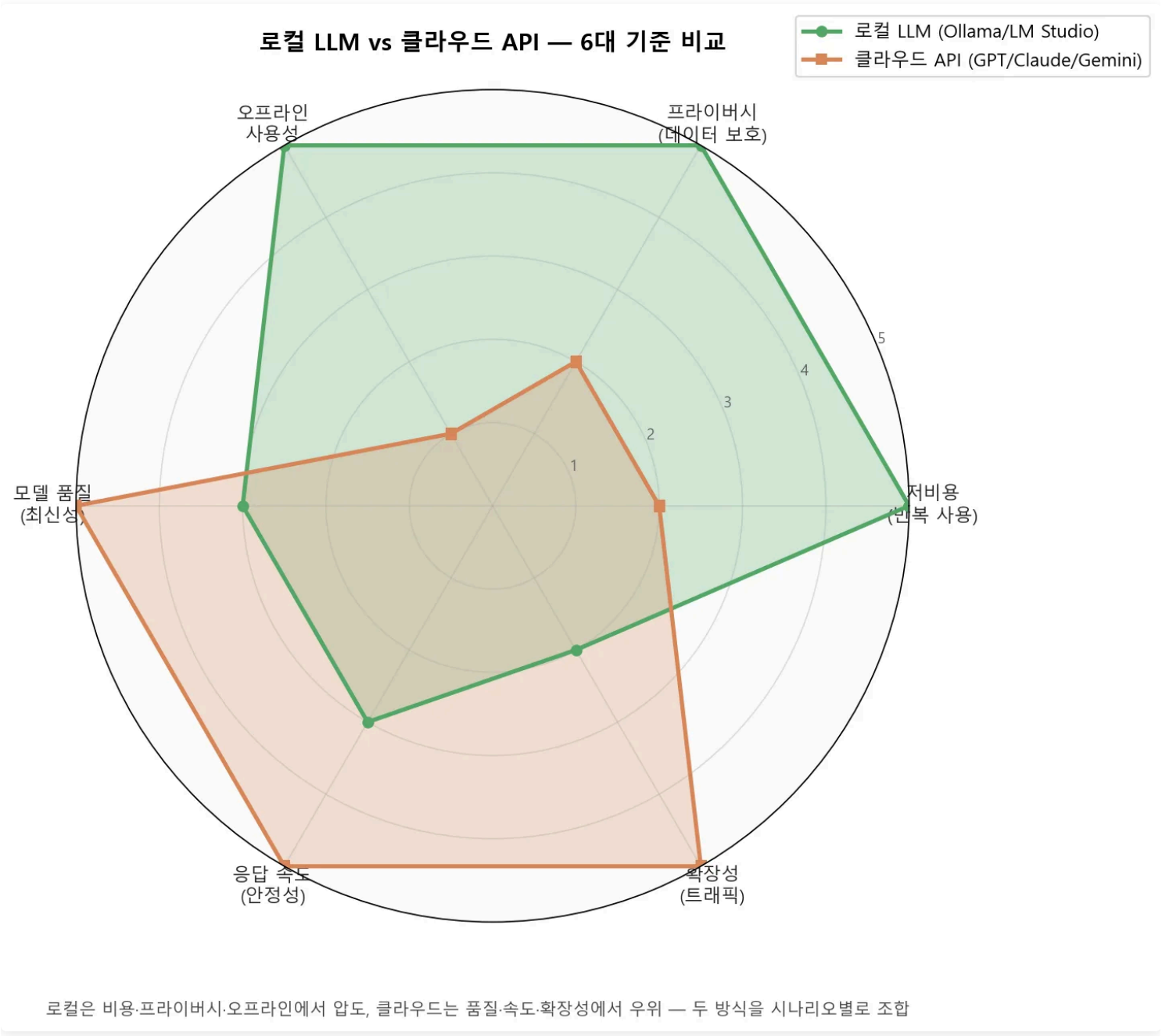
Ollama vs LM Studio 비교

항목	Ollama	LM Studio
인터페이스	CLI (터미널)	GUI (그래픽)
설치 난이도	쉬움	매우 쉬움
모델 탐색	명령어로 검색	시각적 탐색
자동화/스크립트	매우 유리	제한적
API 서버	자동 제공	수동 활성화
주요 사용자	개발자	일반 사용자

25.6 로컬 LLM vs 클라우드 API 비교 (성능, 비용, 속도)

로컬 LLM과 클라우드 API 중 어느 쪽이 더 낫다고 단정하기 어렵습니다. 용도에 따라 최적의 선택이 달라집니다.

종합 비교표



로컬은 비용·프라이버시·오프라인에서 압도, 클라우드는 품질·속도·확장성에서 우위 — 두 방식을 시나리오별로 조합

로컬은 **저비용·프라이버시·오프라인** 축에서 압도적이고, 클라우드는 **모델 품질·응답 속도·확장성** 축에서 우위입니다. 따라서 두 방식 중 하나를 고르기보다 시나리오별(개발/운영, 민감/비민감, 내부/공개)로 조합하는 것이 실전에서 가장 효과적입니다.

비교 항목	로컬 LLM	클라우드 API
비용	초기 하드웨어 비용, 이후 무료	토큰당 과금, 대용량 시 고비용
프라이버시	데이터 외부 전송 없음	제3자 서버에 데이터 전송
오프라인 사용	가능	불가능
모델 품질	중소형 모델 중심	최신·최대 규모 모델
응답 속도	하드웨어에 따라 천차만별	안정적이고 빠름
확장성	하드웨어 한계 존재	무제한 확장
유지보수	직접 관리 필요	제공자가 관리
커스터마이징	파인튜닝 모델 자유롭게 사용	제한적

성능 비교 (2025년 기준 대표 모델)

모델	유형	특징
GPT-4o	클라우드	최고 수준 추론, 멀티모달
Claude 3.5 Sonnet	클라우드	코드·분석 강점
Llama 3.1 70B	로컬 가능	오픈소스 최고 수준, 고사양 필요
Mistral 7B	로컬	효율적, 일상적 작업에 충분
Phi-3 Mini	로컬	소형이지만 뛰어난 추론 능력

> 📌 *[2026 업데이트] 클라우드 대표 모델*
>
> *위 표의 GPT-4o / Claude 3.5 Sonnet은 2024년 세대 기준입니다.* ***2026년 5월 기준 동급 최신****은 다음과 같습니다.*
>
2024년 세대	*2026년 5월 기준*
GPT-4o	***GPT-5*** *(또는 GPT-5.1)*
Claude 3.5 Sonnet	***Claude Sonnet 4.5 / 4.6*** *(1M은 Opus 4.7)*
Llama 3.1 70B	***Llama 4 Maverick / Behemoth***
Phi-3 Mini	***Phi-4 (14B)***
>
> *로컬 LLM도* ***DeepSeek-R1·Qwen 3·Llama 4****처럼 추론·멀티모달이 강한 신규 라인업이 등장하면서, 한층 더 클라우드 API에 근접한 품질을 보이고 있습니다.*

추천 사용 시나리오

로컬 LLM이 적합한 경우

- 반복적인 대용량 텍스트 처리 (비용 절감)
- 개인정보·기밀 데이터 처리
- 인터넷 없는 환경에서의 배포
- 특화된 파인튜닝 모델 활용
- 개발·테스트 단계의 프로토타이핑

클라우드 API가 적합한 경우

- 최고 수준의 응답 품질이 필요할 때
- 트래픽이 불규칙하고 확장성이 필요할 때
- 하드웨어 투자 없이 빠르게 시작할 때
- 멀티모달(이미지·음성) 기능이 필요할 때

25.7 로컬 모델을 기존 앱에 연결하기

로컬 LLM의 진가는 기존 애플리케이션과 연동할 때 발휘됩니다. OpenAI 호환 API 덕분에 기존 코드를 거의 수정하지 않고도 로컬 모델로 전환할 수 있습니다.

LangChain과 연동

LangChain은 Ollama를 공식 지원합니다.

```
from langchain_ollama import OllamaLLM
from langchain_core.prompts import ChatPromptTemplate

# Ollama 모델 초기화
llm = OllamaLLM(model="llama3.2")

# 프롬프트 템플릿 정의
prompt = ChatPromptTemplate.from_messages([
    ("system", "당신은 {topic} 전문가입니다."),
    ("human", "{question}")
])

# 체인 구성
chain = prompt | llm

# 실행
response = chain.invoke({
    "topic": "파이썬",
    "question": "데코레이터 패턴을 설명해주세요."
})
print(response)
```

환경 변수로 API 전환

운영 환경에 따라 클라우드 API와 로컬 LLM을 유연하게 전환하는 패턴입니다.

```python
import os
from openai import OpenAI

# 환경 변수로 백엔드 선택
USE_LOCAL = os.getenv("USE_LOCAL_LLM", "false").lower() == "true"

if USE_LOCAL:
    client = OpenAI(
        base_url="http://localhost:11434/v1",
        api_key="ollama"
    )
    MODEL = "llama3.2"
else:
    client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
    MODEL = "gpt-4o-mini"

def ask(question: str) -> str:
    response = client.chat.completions.create(
        model=MODEL,
        messages=[{"role": "user", "content": question}]
    )
    return response.choices[0].message.content

# USE_LOCAL_LLM=true python app.py 로 로컬 모드 실행
print(ask("파이썬이란 무엇인가요?"))
```

[실습] Ollama로 로컬 챗봇 실행 및 Flask 앱 연동

이번 실습에서는 Ollama로 로컬 LLM을 실행하고, **Flask** 웹 서버와 연동해 간단한 챗봇 웹 애플리케이션을 만듭니다.

실습 목표

- Ollama로 로컬 모델 실행
- Flask API 서버 구축
- 프론트엔드와 백엔드를 연결하는 챗봇 완성

1단계: 환경 준비

```
# 필요 패키지 설치
pip install flask requests

# Ollama 모델 다운로드
ollama pull llama3.2
```

2단계: Flask 백엔드 작성

`app.py` 파일을 생성합니다.

```
from flask import Flask, request, jsonify, render_template_string
import requests
import json

app = Flask(__name__)

# HTML 템플릿 (간결한 인라인 버전)
HTML_TEMPLATE = """
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8">
    <title>로컬 LLM 챗봇</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 700px; margin: 40px auto; padding: 0
20px; }
        #chat-box { border: 1px solid #ddd; height: 400px; overflow-y: auto; padding: 16px;
                    border-radius: 8px; background: #f9f9f9; margin-bottom: 16px; }
        .user-msg { background: #007bff; color: white; padding: 8px 12px; border-radius: 12px;
                    margin: 4px 0; max-width: 80%; margin-left: auto; display: table; }
        .bot-msg { background: #e9ecef; padding: 8px 12px; border-radius: 12px;
                   margin: 4px 0; max-width: 80%; display: table; white-space: pre-wrap; }
        #input-area { display: flex; gap: 8px; }
        #user-input { flex: 1; padding: 10px; border: 1px solid #ddd; border-radius: 8px; font
-size: 16px; }
        button { padding: 10px 20px; background: #007bff; color: white; border: none;
                 border-radius: 8px; cursor: pointer; font-size: 16px; }
        button:hover { background: #0056b3; }
        #status { color: #666; font-size: 14px; margin-top: 8px; }
    </style>
</head>
<body>
    <h2>🤖 로컬 LLM 챗봇 (Ollama)</h2>
    <div id="chat-box"></div>
    <div id="input-area">
        <input type="text" id="user-input" placeholder="메시지를 입력하세요..." />
        <button onclick="sendMessage()">전송</button>
    </div>
    <div id="status"></div>

    <script>
        const chatBox = document.getElementById('chat-box');
```

```
        const userInput = document.getElementById('user-input');
        const status = document.getElementById('status');

        userInput.addEventListener('keypress', (e) => {
            if (e.key === 'Enter') sendMessage();
        });

        function addMessage(text, isUser) {
            const div = document.createElement('div');
            div.className = isUser ? 'user-msg' : 'bot-msg';
            div.textContent = text;
            chatBox.appendChild(div);
            chatBox.scrollTop = chatBox.scrollHeight;
        }

        async function sendMessage() {
            const message = userInput.value.trim();
            if (!message) return;

            addMessage(message, true);
            userInput.value = '';
            status.textContent = '응답 생성 중...';

            try {
                const response = await fetch('/chat', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ message })
                });
                const data = await response.json();
                addMessage(data.response, false);
                status.textContent = `모델: ${data.model} | 완료`;
            } catch (err) {
                addMessage('오류: 서버에 연결할 수 없습니다.', false);
                status.textContent = '오류 발생';
            }
        }
    </script>
</body>
</html>
"""

# 대화 이력 저장 (간단한 메모리 기반)
```

```
conversation_history = []

@app.route('/')
def index():
    """메인 페이지를 반환합니다."""
    return render_template_string(HTML_TEMPLATE)

@app.route('/chat', methods=['POST'])
def chat():
    """사용자 메시지를 받아 Ollama 모델에 전달하고 응답을 반환합니다."""
    data = request.get_json()
    user_message = data.get('message', '').strip()

    if not user_message:
        return jsonify({"error": "메시지가 없습니다."}), 400

    # 대화 이력에 사용자 메시지 추가
    conversation_history.append({
        "role": "user",
        "content": user_message
    })

    # Ollama API 호출 (대화 이력 포함)
    ollama_url = "http://localhost:11434/api/chat"
    payload = {
        "model": "llama3.2",
        "messages": [
            {"role": "system", "content": "당신은 친절하고 도움이 되는 AI 어시스턴트입니다. 한국
어로 답변하세요."}
        ] + conversation_history,
        "stream": False
    }

    try:
        response = requests.post(ollama_url, json=payload, timeout=120)
        response.raise_for_status()

        result = response.json()
        bot_message = result["message"]["content"]

        # 대화 이력에 어시스턴트 응답 추가
        conversation_history.append({
            "role": "assistant",
```

```
            "content": bot_message
        })

        return jsonify({
            "response": bot_message,
            "model": result.get("model", "llama3.2")
        })

    except requests.exceptions.ConnectionError:
        return jsonify({"error": "Ollama 서버에 연결할 수 없습니다. 'ollama serve'를 실행하세
요."}), 503
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route('/reset', methods=['POST'])
def reset():
    """대화 이력을 초기화합니다."""
    conversation_history.clear()
    return jsonify({"message": "대화 이력이 초기화되었습니다."})

if __name__ == '__main__':
    print("로컬 LLM 챗봇 서버를 시작합니다...")
    print("브라우저에서 http://localhost:5000 을 열어주세요.")
    print("Ollama가 실행 중인지 확인하세요: ollama serve")
    app.run(debug=True, port=5000)
```

3단계: 실행

터미널 두 개를 열어 각각 실행합니다.

```
# 터미널 1: Ollama 서버 확인 (이미 실행 중이면 생략)
ollama serve

# 터미널 2: Flask 앱 실행
python app.py
```

브라우저에서 `http://localhost:5000` 을 열면 챗봇 인터페이스가 나타납니다.

4단계: 동작 확인

```
# API 직접 테스트
curl -X POST http://localhost:5000/chat \
     -H "Content-Type: application/json" \
     -d '{"message": "파이썬의 특징을 알려주세요."}'
```

예상 응답:

```
{
    "response": "파이썬은 가독성이 높고 배우기 쉬운 프로그래밍 언어입니다...",
    "model": "llama3.2"
}
```

5단계: 모델 교체 실험

`app.py`에서 모델명만 바꾸면 다른 모델을 사용할 수 있습니다.

```
# llama3.2 → mistral 또는 phi3:mini 로 변경
payload = {
    "model": "mistral",  # 이 부분만 수정
    ...
}
```

> ***성능 비교**: 동일한 질문으로 여러 모델을 테스트하고 응답 품질과 속도를 비교해보세요. 모델마다 특성이 다름을 직접 체감할 수 있습니다.*

핵심 정리

개념	설명
로컬 LLM	외부 서버 없이 자신의 하드웨어에서 실행하는 언어 모델
양자화	모델 가중치를 압축해 메모리 사용량을 줄이는 기법
GGUF	llama.cpp 기반의 효율적인 모델 파일 포맷
Ollama	로컬 LLM을 CLI로 손쉽게 실행하는 도구
LM Studio	GUI 기반의 로컬 LLM 실행 및 관리 도구
OpenAI 호환 API	기존 OpenAI SDK 코드를 그대로 재사용 가능한 인터페이스

이 챕터에서는 **Ollama**와 **LM Studio**를 통해 로컬 LLM을 실행하고, REST API와 Flask를 연동해 실제 동작하는 챗봇을 만들었습니다. 다음 챕터에서는 지금까지 배운 모든 내용을 종합해 완성도 있는 AI 프로젝트를 구축합니다.

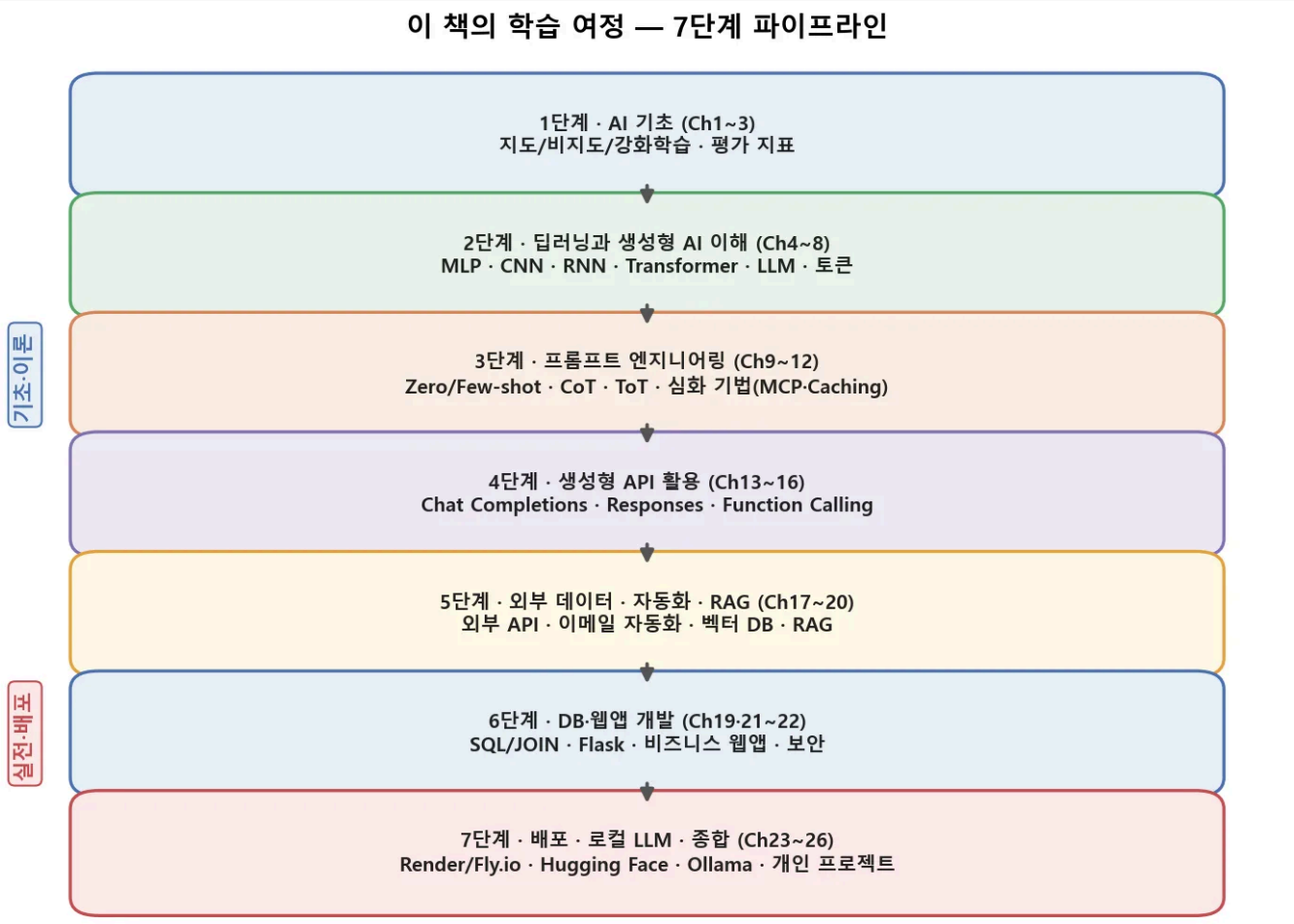
> *이전: Ch.24 Hugging Face 모델 활용 | 다음: Ch.26 종합 정리와 프로젝트*

이 책의 마지막 챕터입니다. 지금까지 AI 기초부터 시작해 딥러닝, 생성형 AI, 프롬프트 엔지니어링, API 활용, RAG, 웹 애플리케이션 개발, 그리고 로컬 LLM 실행까지 긴 여정을 함께했습니다. 이 챕터에서는 전체 과정을 되돌아보고, 앞으로의 방향을 설계하며, 여러분만의 개인 프로젝트를 완성하는 것을 목표로 합니다.

26.1 전체 과정 핵심 리뷰

지금까지 배운 내용을 하나의 흐름으로 정리합니다. 각 챕터는 독립된 주제처럼 보이지만, 실제로는 하나의 큰 파이프라인을 이루고 있습니다.

학습 흐름 전체 지도



각 단계는 독립적이지 않고 다음 단계의 디딤돌 — 7단계를 모두 거치면 "기획→구현→배포"의 풀스택 사이클을 갖춤

각 단계는 독립된 주제처럼 보이지만 다음 단계의 디딤돌이 됩니다. 7단계를 모두 거치면 단순한 모델 호출을 넘어 **기획 → 구현 → 배포**까지 이어지는 풀스택 사이클을 갖추게 됩니다.

단계별 핵심 요약

1단계 — AI 기초 (Ch.01~03)

- **머신러닝(Machine Learning)** 의 세 가지 학습 방식: 지도, 비지도, 강화학습
- **딥러닝(Deep Learning)** 은 머신러닝의 하위 분야로, 다층 신경망을 활용합니다.
- 데이터 전처리와 모델 평가 지표(정확도, F1-score, RMSE)의 의미

2단계 — 생성형 AI 이해 (Ch.04~08)

- **거대 언어 모델(LLM, Large Language Model)** 의 작동 원리: 트랜스포머 아키텍처, 어텐션 메커니즘
- **토큰(Token)** 이란 LLM이 텍스트를 처리하는 최소 단위입니다.
- **파인튜닝(Fine-tuning)** 과 **프롬프팅(Prompting)** 의 차이: 파인튜닝은 모델 가중치를 조정하고, 프롬프팅은 입력을 조정합니다.

3단계 — 프롬프트 엔지니어링 (Ch.09~12)

- **제로샷(Zero-shot)**, **퓨샷(Few-shot)**, **체인오브소트(Chain-of-Thought)** 프롬프팅 기법
- 역할 부여(Role Prompting), 출력 형식 지정, 제약 조건 설정의 실전 활용
- 프롬프트 평가 방법: 일관성, 정확성, 유용성 기준

4단계 — API 활용 (Ch.13~16)

- **OpenAI API** 의 핵심 엔드포인트: `chat/completions`, `embeddings`, `images/generations`
- **스트리밍(Streaming)** 응답으로 사용자 경험 개선
- 비용 최적화: 토큰 수 관리, 모델 선택 전략

5단계 — RAG 구현 (Ch.17~20)

- **RAG(Retrieval-Augmented Generation)** 의 구조: 문서 청킹 → 임베딩 → 벡터 저장 → 유사도 검색 → 생성
- **벡터 데이터베이스(Vector Database)**: Chroma, Pinecone, FAISS 비교
- **임베딩(Embedding)** 의 품질이 검색 성능을 결정합니다.

6단계 — 웹 앱 개발 (Ch.21~24)

- **Streamlit** 으로 빠른 프로토타입 제작, **FastAPI** 로 프로덕션 수준 API 서버 구축
- **랭체인(LangChain)** / **LlamaIndex** 프레임워크를 활용한 파이프라인 구성
- 세션 관리, 사용자 인증, 에러 핸들링의 실전 패턴

7단계 — 로컬 LLM (Ch.25)

- **Ollama** 를 사용해 인터넷 연결 없이 모델을 실행합니다.
- **GGUF** 포맷과 양자화(Quantization)로 일반 PC에서도 실행 가능합니다.
- 개인정보 보호, 비용 절감, 커스터마이징 가능성이 핵심 장점입니다.

전체 기술 스택 요약표

영역	주요 기술/도구	핵심 개념
AI 기초	Python, NumPy, scikit-learn	지도학습, 평가지표
딥러닝	PyTorch, TensorFlow	신경망, 역전파
생성형 AI	GPT-4, Claude, Gemini	트랜스포머, 토큰
프롬프트	프롬프트 엔지니어링 기법	CoT, 퓨샷, 역할부여
API	OpenAI API, Anthropic API	REST, 스트리밍
RAG	LangChain, Chroma, FAISS	임베딩, 벡터검색
웹앱	Streamlit, FastAPI	프론트엔드, 백엔드
로컬 LLM	Ollama, llama.cpp	GGUF, 양자화

26.2 생성형 AI 기술 트렌드와 전망

생성형 AI 분야는 매우 빠르게 변화하고 있습니다. 현재 가장 주목받는 세 가지 방향을 살펴봅니다.

AI 에이전트 (AI Agent)

AI 에이전트(AI Agent) 는 단순히 질문에 답하는 것을 넘어, 스스로 계획을 세우고 도구를 사용하여 복잡한 작업을 수행하는 시스템입니다.

> *에이전트는 "지시를 따르는 AI"가 아니라 "목표를 향해 스스로 행동하는 AI"입니다. 사용자가 "여행 계획을 짜줘"라고 하면, 에이전트는 웹 검색, 일정 확인, 예약 시스템 호출 등을 자율적으로 수행합니다.*

핵심 구성 요소:

- **계획(Planning)**: 큰 목표를 작은 하위 작업으로 분해합니다.

- **메모리(Memory)**: 단기(대화 히스토리)와 장기(외부 데이터베이스) 기억을 활용합니다.
- **도구 사용(Tool Use)**: 웹 검색, 코드 실행, API 호출 등 외부 도구를 활용합니다.
- **행동(Action)**: 계획에 따라 실제 작업을 실행합니다.

주요 에이전트 프레임워크:

프레임워크	특징	적합한 용도
LangGraph	상태 그래프 기반, 복잡한 워크플로	다단계 에이전트 파이프라인
AutoGen	멀티 에이전트 대화 방식	복수 AI 간 협업
CrewAI	역할 기반 에이전트 팀	전문화된 에이전트 조합
Claude Agent SDK	Anthropic 공식 에이전트 SDK	Claude 기반 에이전트 개발

멀티모달 AI (Multimodal AI)

멀티모달(Multimodal) AI는 텍스트, 이미지, 음성, 영상 등 여러 형태의 입력을 통합적으로 이해하고 생성합니다.

현재 주요 멀티모달 능력:

- 이미지를 보고 텍스트로 설명하기 (Image Captioning)
- 텍스트 설명으로 이미지 생성하기 (Text-to-Image)
- 음성을 텍스트로, 텍스트를 음성으로 변환하기 (STT/TTS)
- 영상을 분석하고 요약하기 (Video Understanding)

GPT-4o, **Gemini 1.5 Pro**, **Claude 3.5 Sonnet** 등은 이미 텍스트와 이미지를 함께 처리합니다. 앞으로는 실시간 음성·영상 상호작용이 일반화될 것으로 전망됩니다.

> 📌 *[2026 업데이트] 멀티모달 라인업*
>
> *2026년 5월 기준 동급 최신 멀티모달 모델은 다음과 같습니다.*

운영사	2024년 세대	2026년 5월 기준 동급 최신
OpenAI	GPT-4o	GPT-5 (Responses API), o3
Google	Gemini 1.5 Pro	Gemini 2.5 Pro / Gemini 3 Pro (Deep Think)
Anthropic	Claude 3.5 Sonnet	Claude Sonnet 4.5 / 4.6, Opus 4.7 (1M 컨텍스트)
오픈소스	Llama 3.1 / Qwen 2.5	Llama 4 (네이티브 멀티모달), Qwen 3

> *또한 추론 특화 모델(o3·o4-mini, Claude Opus 4.7 Extended Thinking, **DeepSeek-R1**)이 별도 라인업으로 자리 잡았습니다.*

소형 언어 모델 (SLM, Small Language Model)

SLM(Small Language Model) 은 수십억 파라미터 규모의 작은 모델로, 특정 영역에 특화되어 높은 성능을 냅니다.

> *"작지만 강하다" — SLM은 모든 것을 잘하는 거대 모델 대신, 특정 분야에서 GPT-4 수준의 성능을 훨씬 적은 자원으로 달성하는 것을 목표로 합니다.*

SLM의 장점:

- 엣지 디바이스(스마트폰, IoT)에서 실행 가능
- 응답 속도가 빠르고 비용이 낮음
- 특정 도메인 데이터로 파인튜닝 시 뛰어난 성능
- 개인정보 보호 강화 (데이터가 외부로 나가지 않음)

주목할 SLM 모델들:

모델	개발사	파라미터	특징
Phi-3 Mini	Microsoft	3.8B	교육 데이터 중심 학습
Gemma 2 2B	Google	2B	오픈소스, 경량
Llama 3.2 3B	Meta	3B	멀티모달 지원
Qwen 2.5 1.5B	Alibaba	1.5B	다국어, 코드 특화

앞으로의 전망

- **개인화 AI**: 사용자 개인의 데이터와 선호를 학습한 나만의 AI 어시스턴트
- **AI 네이티브 앱**: 처음부터 AI를 핵심으로 설계된 애플리케이션
- **추론 특화 모델**: o1, o3, DeepSeek-R1처럼 복잡한 수학·논리 문제를 단계적으로 추론하는 모델
- **자율 에이전트 생태계**: 여러 AI 에이전트가 협력하는 분산 AI 시스템

26.3 AI 윤리와 책임 있는 AI 사용

기술을 배우는 것만큼 중요한 것이 그 기술을 어떻게 사용하느냐입니다.

생성형 AI의 주요 윤리 이슈

1. 환각(Hallucination) 문제

LLM은 사실이 아닌 내용을 마치 사실인 것처럼 생성할 수 있습니다. 이를 **환각(Hallucination)** 이라고 합니다.

> *AI가 생성한 정보를 그대로 신뢰하지 마십시오. 특히 의료, 법률, 금융 분야에서는 반드시 전문가의 검증이 필요합니다.*

대응 방법:

- RAG를 활용해 신뢰할 수 있는 문서 기반으로 답변 생성
- 출처 명시를 프롬프트에 요구
- 중요한 정보는 외부 소스에서 직접 확인

2. 편향(Bias) 문제

LLM은 학습 데이터에 존재하는 편향을 그대로 학습합니다. 성별, 인종, 문화적 편향이 출력에 반영될 수 있습니다.

대응 방법:

- 다양한 관점을 요청하는 프롬프트 설계
- 출력 결과의 편향 여부를 주기적으로 검토
- 편향 탐지 도구 활용

3. 저작권과 데이터 프라이버시

- AI가 생성한 콘텐츠의 저작권 귀속은 아직 법적으로 명확하지 않습니다.
- 개인정보가 포함된 데이터를 외부 API에 전송하는 것은 개인정보보호법 위반 소지가 있습니다.
- 기업 기밀을 공개 AI 서비스에 입력하지 않도록 주의합니다.

4. AI 생성 콘텐츠 표시 의무

AI가 작성하거나 생성한 콘텐츠임을 명시하는 것이 투명성과 신뢰를 높입니다. 일부 국가와 플랫폼에서는 이를 의무화하고 있습니다.

책임 있는 AI 개발 원칙

원칙	내용
투명성(Transparency)	AI 사용 여부와 작동 방식을 명확히 공개합니다.
공정성(Fairness)	특정 집단에 불이익을 주는 편향을 최소화합니다.
안전성(Safety)	악의적 사용을 방지하는 가드레일을 설계합니다.
책임성(Accountability)	AI 출력의 결과에 대한 책임 소재를 명확히 합니다.
프라이버시(Privacy)	개인정보를 수집·처리할 때 동의와 최소화 원칙을 따릅니다.

실천 체크리스트

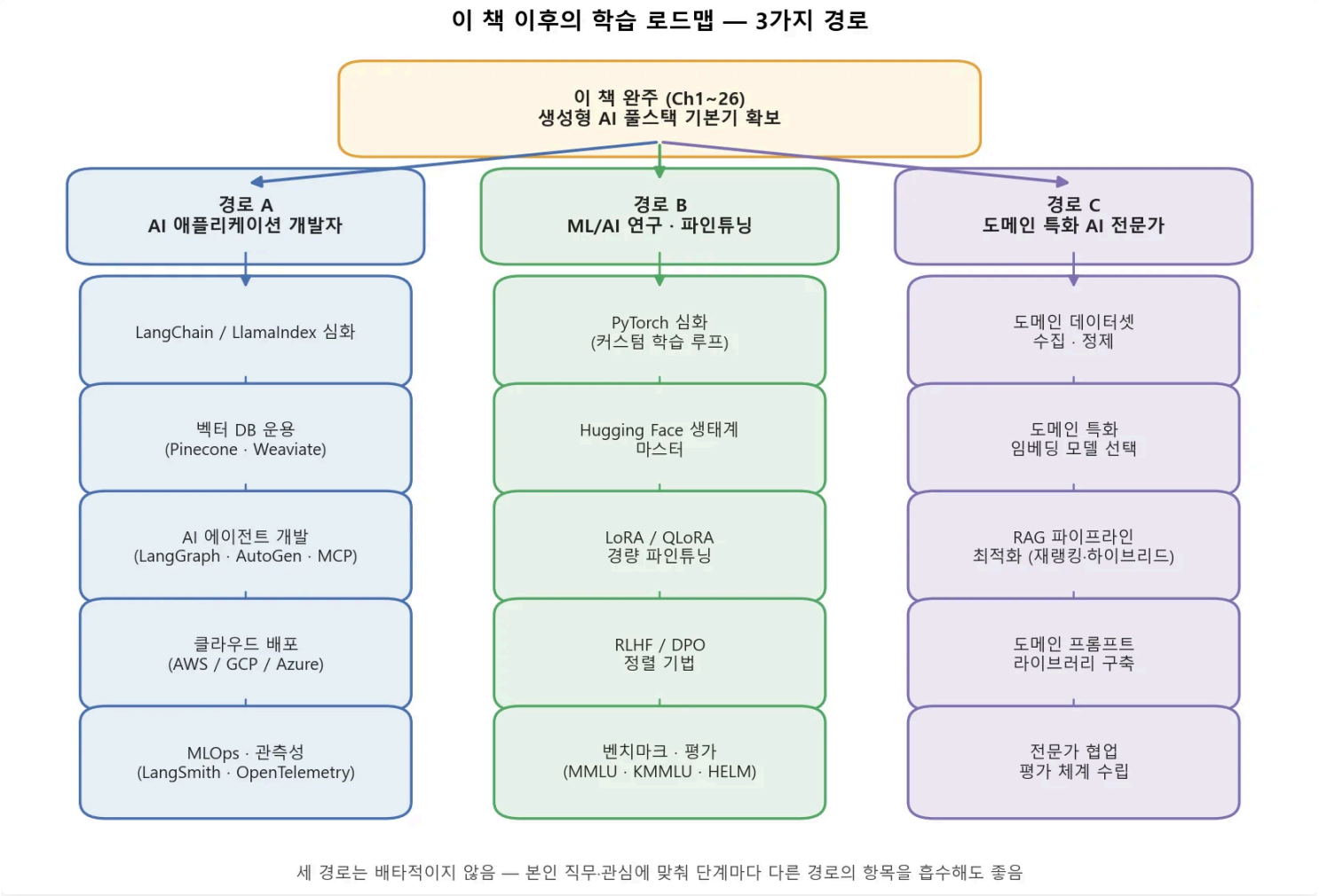
다음 항목을 개발 시 점검해 보시기 바랍니다.

- [ ] AI 생성 결과물에 대한 사람의 최종 검토 절차가 있습니까?
- [ ] 사용자에게 AI와 대화하고 있음을 알립니까?
- [ ] 민감한 개인정보가 외부 AI API로 전송되지 않습니까?
- [ ] 시스템 프롬프트에 유해 콘텐츠 생성 방지 지침이 포함되어 있습니까?

- [ ] 오류나 편향된 출력에 대한 피드백 채널이 있습니까?

26.4 학습 로드맵 — 이후 어떻게 공부할 것인가

이 책을 마친 후 어떤 방향으로 나아갈지 목표에 따라 로드맵을 제안합니다.



경로 A(앱 개발자), **경로 B(연구·파인튜닝)**, **경로 C(도메인 특화)** 세 갈래로 정리했습니다. 세 경로는 배타적이지 않으니, 본인 직무·관심에 맞춰 단계마다 다른 경로의 항목도 함께 흡수하면 좋습니다.

목표별 추천 학습 경로

경로 A — AI 애플리케이션 개발자

실용적인 AI 서비스를 만들고 싶은 분에게 적합합니다.

```
현재 수준
    ↓
LangChain / LlamaIndex 심화
    ↓
벡터 데이터베이스 운용 (Pinecone, Weaviate)
    ↓
AI 에이전트 개발 (LangGraph, AutoGen)
    ↓
클라우드 배포 (AWS, GCP, Azure)
    ↓
MLOps 기초 (모델 모니터링, A/B 테스트)
```

경로 B — ML/AI 연구 및 파인튜닝

모델을 직접 훈련하고 개선하고 싶은 분에게 적합합니다.

```
현재 수준
    ↓
PyTorch 심화 (커스텀 데이터로더, 훈련 루프)
    ↓
허깅페이스(Hugging Face) 생태계 마스터
    ↓
LoRA / QLoRA 파인튜닝
    ↓
RLHF / DPO 정렬 기법
    ↓
모델 평가 및 벤치마크
```

경로 C — 도메인 특화 AI 전문가

특정 산업(의료, 법률, 금융, 교육)에 AI를 적용하고 싶은 분에게 적합합니다.

```
현재 수준
    ↓
도메인 특화 데이터셋 수집·정제
    ↓
도메인 특화 임베딩 모델 선택
    ↓
RAG 파이프라인 최적화
    ↓
도메인 용어 기반 프롬프트 라이브러리 구축
    ↓
도메인 전문가와 협업하여 평가 체계 수립
```

26.5 개인 프로젝트 진행

이 책에서 배운 기술을 활용해 직접 프로젝트를 완성해 봅니다. 실제로 만들어보는 것이 가장 강력한 학습 방법입니다.

주제 선정 가이드라인

좋은 프로젝트 주제의 조건은 다음과 같습니다.

1. 나에게 의미 있는 문제를 선택합니다.

> *관심 없는 주제는 오래 지속하기 어렵습니다. 일상에서 불편함을 느끼거나 반복적으로 하는 작업 중에서 AI로 해결할 수 있는 것을 찾습니다.*

2. 범위를 명확히 제한합니다.

처음부터 큰 시스템을 만들려 하면 완성하지 못합니다. **MVP(Minimum Viable Product)**, 즉 핵심 기능만 담은 최소 기능 제품으로 시작합니다.

3. 기술 스택을 현재 수준에 맞춥니다.

수준	추천 기술 스택
입문	Streamlit + OpenAI API
중급	FastAPI + LangChain + Chroma
고급	FastAPI + LangGraph + Pinecone + 클라우드 배포

프로젝트 아이디어 예시:

아이디어	핵심 기술	예상 구현 기간
나만의 논문 요약봇	RAG + PDF 파싱	1~2주
고객 리뷰 감성 분석 대시보드	LLM + Streamlit	1주
회사 내부 문서 Q&A 챗봇	RAG + FastAPI	2~3주
코드 리뷰 자동화 도구	OpenAI API + GitHub API	2주
개인 일기 분석 및 감정 트래킹	LLM + 데이터 시각화	2주
특정 도메인 뉴스 요약 뉴스레터	LLM + 스케줄러	1~2주

프로젝트 기획서 작성법

프로젝트를 시작하기 전에 반드시 기획서를 작성합니다. 한 페이지 분량으로 충분합니다.

기획서 필수 항목:

```
1. 프로젝트 제목
   - 한 문장으로 설명할 수 있는 이름

2. 문제 정의 (Problem Statement)
   - 어떤 문제를 해결하는가?
   - 왜 이 문제가 중요한가?

3. 대상 사용자 (Target User)
   - 누가 이 서비스를 사용하는가?

4. 핵심 기능 (Core Features) — 최대 3가지
   - 기능 1: 설명
   - 기능 2: 설명
   - 기능 3: 설명

5. 기술 스택 (Tech Stack)
   - LLM 모델: (예: GPT-4o-mini)
   - 프레임워크: (예: LangChain)
   - 프론트엔드: (예: Streamlit)
   - 데이터 저장: (예: Chroma)

6. 성공 지표 (Success Metrics)
   - 어떻게 되었을 때 성공인가?

7. 일정 (Timeline)
   - 1주차: 설계 및 환경 구축
   - 2주차: 핵심 기능 구현
   - 3주차: 테스트 및 개선
   - 4주차: 배포 및 발표 준비
```

구현 → 테스트 → 배포 사이클

구현 단계:

```
# 개발 우선순위 원칙
# 1. 먼저 동작하게 (Make it work)
# 2. 그다음 올바르게 (Make it right)
# 3. 마지막으로 빠르게 (Make it fast)
```

- 핵심 기능부터 구현하고, 나머지는 나중에 추가합니다.

- 하루에 하나씩 작은 기능을 완성하는 방식으로 진행합니다.
- 코드를 짤 때마다 Git 커밋으로 기록을 남깁니다.

테스트 단계:

테스트 유형	확인 내용	도구
기능 테스트	각 기능이 의도대로 작동하는가	수동 테스트
엣지 케이스	비정상 입력, 빈 입력 처리	pytest
LLM 출력 품질	답변의 정확성과 유용성	직접 평가
성능 테스트	응답 속도, 토큰 사용량	타이머 측정

배포 단계:

초보자를 위한 추천 배포 옵션:

플랫폼	특징	비용
Streamlit Cloud	Streamlit 앱 무료 배포, GitHub 연동	무료
Hugging Face Spaces	AI 앱 특화, 간단한 배포	무료
Railway	백엔드 API 서버 배포	소액
Vercel	프론트엔드 중심, 서버리스	무료 티어 있음

Streamlit Cloud 배포 기본 절차:

```
1. GitHub 저장소 생성 및 코드 업로드
2. requirements.txt 작성
3. .streamlit/secrets.toml에 API 키 설정 (로컬용)
4. Streamlit Cloud에서 GitHub 저장소 연결
5. Secrets 탭에서 환경변수 설정
6. Deploy 클릭
```

26.6 프로젝트 발표 및 피드백

프로젝트를 완성했다면, 다른 사람에게 발표하고 피드백을 받는 과정이 매우 중요합니다.

발표 구성 가이드

발표는 10분 기준으로 다음과 같이 구성합니다:

파트	시간	내용
문제 제기	1분	왜 이 프로젝트를 만들었는가?
데모 시연	4분	실제 작동하는 화면을 보여줍니다.
기술 설명	3분	어떤 기술을 어떻게 활용했는가?
배운 점과 한계	1분	무엇을 배웠고, 무엇이 아쉬운가?
Q&A	1분	질문에 답합니다.

> *발표에서 가장 중요한 것은 "완벽한 코드"가 아니라 "문제를 이해하고 해결하려 한 과정"입니다. 실패와 시행착오도 훌륭한 발표 내용이 됩니다.*

피드백 받는 법

좋은 피드백을 이끌어내는 질문:

- "이 기능 중 가장 유용하다고 생각하는 것은 무엇입니까?"
- "사용하면서 불편하거나 헷갈리는 부분이 있었습니까?"
- "이 서비스가 실제로 도움이 될 것 같습니까? 그렇다면/아니라면 왜입니까?"
- "추가하면 좋겠다고 생각하는 기능이 있습니까?"

피드백을 받을 때 주의할 점:

1. 방어적으로 반응하지 않습니다. 비판은 개선의 기회입니다.
2. 모든 피드백을 다 반영할 필요는 없습니다. 프로젝트의 방향에 맞는 것을 선택합니다.
3. 피드백을 기록해 두고, 우선순위를 정해 다음 버전에 반영합니다.

프로젝트 이후 성장 방법

- **포트폴리오 페이지** 를 만들어 프로젝트를 문서화합니다.
- **GitHub README** 에 프로젝트 설명, 설치 방법, 데모 링크를 작성합니다.
- **기술 블로그** 에 개발 과정에서 만난 문제와 해결법을 공유합니다.
- 피드백을 반영해 **버전 2.0** 으로 발전시킵니다.

[최종 프로젝트] 기획 → 구현 → 발표

이 책에서 배운 모든 내용을 통합한 최종 프로젝트를 완성합니다.

최종 프로젝트 요구사항

필수 포함 요소:

- [ ] LLM API 또는 로컬 LLM 활용
- [ ] RAG 파이프라인 구현 (외부 문서 또는 데이터 활용)
- [ ] 웹 인터페이스 (Streamlit 또는 FastAPI + 프론트엔드)
- [ ] 실제 문제를 해결하는 유용성
- [ ] GitHub 저장소에 코드 공개 및 README 작성

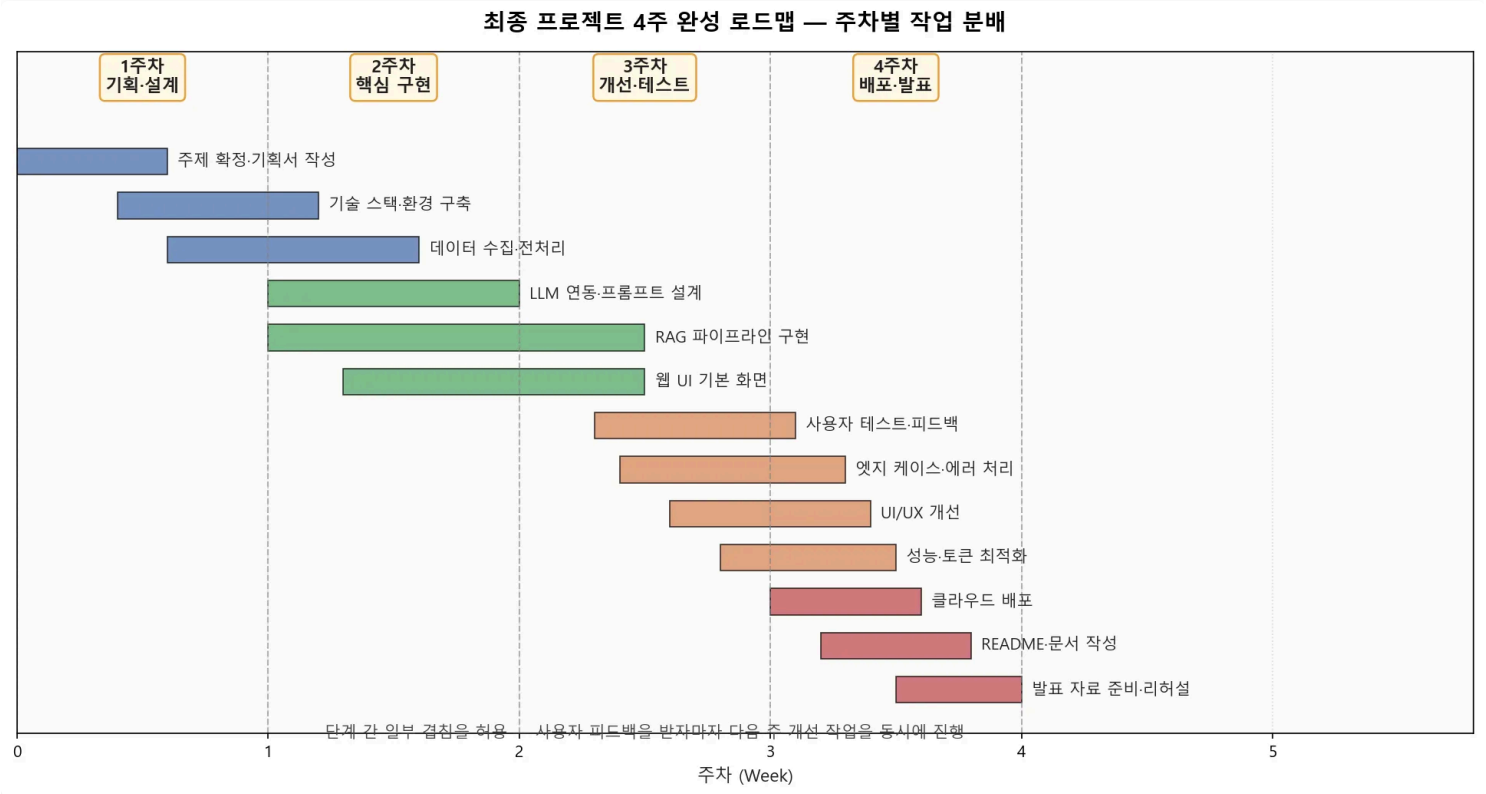
선택 요소 (추가 시 가산점):

- [ ] 클라우드 배포 (실제 접속 가능한 URL)
- [ ] 사용자 인증 및 대화 히스토리 저장
- [ ] 에이전트 기능 (도구 사용, 다단계 추론)
- [ ] 멀티모달 입력 처리 (이미지, 음성)
- [ ] 로컬 LLM 옵션 지원

최종 프로젝트 평가 기준

항목	배점	평가 기준
문제 정의의 명확성	20점	해결하려는 문제가 구체적이고 의미 있는가?
기술 구현 완성도	30점	핵심 기능이 안정적으로 작동하는가?
사용자 경험	20점	직관적이고 사용하기 편리한가?
코드 품질	15점	구조가 명확하고 재사용 가능한가?
발표 및 설명	15점	프로젝트를 명확하게 설명하는가?

4주 완성 로드맵



주차별로 단계가 깔끔하게 나뉘는 듯 보이지만, 실제로는 단계 간 약간의 겹침을 허용해야 합니다. 예를 들어 3주차에 받은 사용자 피드백을 곧바로 4주차 개선에 반영하는 식으로 짧은 반복 사이클을 운영하면 마감 직전 폭주를 피할 수 있습니다.

1주차 — 기획 및 설계

- 프로젝트 주제 확정 및 기획서 작성
- 기술 스택 결정 및 개발 환경 구축

- 데이터 수집 및 전처리 시작
- GitHub 저장소 생성, 브랜치 전략 수립

2주차 — 핵심 기능 구현

- LLM 연동 및 프롬프트 설계
- RAG 파이프라인 구현 (문서 청킹, 임베딩, 벡터 저장)
- 웹 인터페이스 기본 화면 구현
- 일일 커밋으로 진행 상황 기록

3주차 — 개선 및 테스트

- 사용자 테스트 및 피드백 수집
- 엣지 케이스 처리 및 에러 핸들링
- UI/UX 개선
- 성능 최적화 (토큰 수, 응답 속도)

4주차 — 배포 및 발표 준비

- 클라우드 배포 (Streamlit Cloud, HuggingFace Spaces 등)
- README 및 문서 작성
- 발표 자료 준비
- 최종 점검 및 발표

마치며

긴 여정을 마쳤습니다.

처음 AI가 무엇인지 막연한 궁금증을 품고 이 책을 펼쳤던 순간부터, 지금 이 마지막 페이지에 도달하기까지 여러분은 정말 많은 것을 배웠습니다.

머신러닝의 기초 개념을 이해하고, 딥러닝의 원리를 파악하고, 생성형 AI가 어떻게 작동하는지 배웠습니다. 프롬프트를 설계하고, API를 호출하고, 문서를 벡터로 변환하고, 그것을 검색해 답변을 생성하는 RAG 파이프라인을 직접 만들었습니다. 웹 애플리케이션으로 실제 서비스처럼 포장하고, 인터넷 없이도 내 컴퓨터에서 모델을 실행하는 방법까지 익혔습니다.

이 모든 것이 1~2년 전만 해도 AI 전문가들의 전유물이었습니다. 지금 여러분은 그 영역에 발을 내딛었습니다.

> *중요한 것은 "완벽하게 이해했는가"가 아닙니다. "직접 만들어봤는가"입니다. 코드를 한 줄이라도 직접 실행해보고, 에러를 만나고, 그것을 해결해본 경험이 진짜 배움입니다.*

AI 기술은 계속 발전합니다. 오늘 배운 것이 6개월 뒤에는 바뀔 수 있습니다. 하지만 변하지 않는 것이 있습니다. 문제를 정의하는 능력, 적절한 도구를 찾아 적용하는 능력, 그리고 배운 것을 실제로 만들어내는 능력입니다. 이 책을 통해 그 기반을 닦았습니다.

이제 여러분의 차례입니다. 작은 프로젝트 하나를 시작해 보십시오. 완벽하지 않아도 됩니다. 누군가에게 유용하지 않아도 됩니다. 그냥 만들어보십시오. 그 경험이 다음 프로젝트를, 그리고 그다음 성장을 이끌어 줄 것입니다.

이 책을 끝까지 함께해 주셔서 진심으로 감사합니다. 여러분의 AI 여정은 이제 막 시작되었습니다.

이전 챕터: Chapter 25. 로컬 LLM 실행